

Cavefish: Communication-Optimal Light Client Protocol for UTxO Ledgers

AGGELOS KIAYIAS, The University of Edinburgh, UK and Input Output

MARC ROESCHLIN, POLINA VINOGRADOVA, and PYRROS CHAIDOS, Input Output

Blockchain light clients (LCs) are agents with limited resources that are not able or not willing to maintain a fully validated copy of the ledger. They rely on service providers (SPs), typically full nodes, to access data required for tasks such as constructing transactions or interacting with off-chain applications.

We introduce Cavefish, a novel protocol for UTxO-based platforms that enables LCs to submit transactions with minimal trust, storage, and computation overheads. Cavefish defines a two-party computation protocol between an LC and an SP, in which the LC specifies a transaction and the SP constructs it. The LC only receives a blinded version of the transaction, preventing modification and reuse, but allowing the LC to verify the transaction against the original intent. The SP is compensated inside the constructed transaction, eliminating the need for a separate protocol or exchange.

To support this, we propose a variant of the predicate blind signature (PBS) scheme of Fuchsbaier and Wolf (Eurocrypt 2024), letting the SP obtain a valid signature on the unblinded transaction, which can then be posted on chain as the resulting signatures verify as standard Schnorr signatures. When combined with hierarchical deterministic (HD) wallets, the LC can provide a single public key and chain code to the SP, reducing communication footprint to a minimum. To further reduce overheads, our PBS variant relaxes the unlinkability requirements of traditional blind signatures in favor of efficiency as transactions only need to stay private until posted.

On top of this, we define a multi-SP version of Cavefish that provides strong liveness and optimality guarantees. We benchmark the Non-interactive Argument of Knowledge (NArg) component of Cavefish on two major UTxO-based blockchains. Our results show that proving and verification times, as well as circuit sizes, are practical for real-world deployment.

1 Introduction

Blockchains are distributed append-only data structures that grow continuously over time. They offer strong guarantees of data integrity, censorship resistance, and fault tolerance through cryptographic protocols and distributed consensus. Full nodes of most blockchains are required to download and validate the entire chain history to ensure correctness and security. This presents a significant barrier for resource-constrained devices, such as smartphones and embedded systems.

Light client (LC) protocols aim to mitigate this issue by enabling nodes to interact with the blockchain in a secure and efficient manner without maintaining full historical data. These protocols must strike a careful balance between minimizing resource consumption and preserving critical security properties, most notably, validated information about the current chain state. As the chain length increases, it becomes prohibitively expensive for a light client to scan the entire history to verify past transactions. In this work, we explore how an LC with maximally constrained resources (state storage and communication) can securely construct a valid transaction with the help of a compensated service provider.

We rely on the *Unspent Transaction Output* (UTxO [42] [35]) ledger model for managing asset ownership and validating transactions, which requires its users to keep track of multiple past transactions rather than a single account. This paradigm makes it harder for a resource-constrained client to keep track of parts of the chain state that are relevant to its operation (e.g. transaction construction). It does, however, provide more robust formal security guarantees,

Authors' Contact Information: Aggelos Kiayias, firstname.lastname@iohk.io, The University of Edinburgh, Edinburgh, UK and Input Output; Marc Roeschlin; Polina Vinogradova; Pyrros Chaidos, firstname.lastname@iohk.io, Input Output.

especially regarding transaction validation. These guarantees allow us to formulate *Cavefish*, a novel secure intent-based light client protocol designed for UTxO-based blockchains.

Cavefish enables light clients with minimal (local) storage and computation capacity to submit transactions without downloading or querying any part of the current ledger or chain state. It is a two-party computation protocol that yields a signed transaction identical to one created by a full node. In addition to the low storage and computational requirements, our protocol is communication-optimal, and can be completed in as few as two rounds following the initial intent submission. The only state the LC is expected to maintain is the public key address(es) of its funds. Additionally, Cavefish is compatible with hierarchical (HD) wallets [57] allowing for easy address discovery over a range of child addresses with the LC sending only a single public key together with its chain code.

1.1 Protocol Overview

The protocol is initiated by the LC who sends an intent and one or more of their previously used addresses to the service provider. An *intent* is a structured description of what a transaction should accomplish. The SP builds a transaction that satisfies the intent, with the input funds coming from the address specified by the LC.

If the SP were to construct the intent-fulfilling transaction and then share it in the clear, a freeloading LC could sign and submit the transaction to the chain on its own, evading the SP's reimbursement. Hiding the transaction is what enforces compensation to the SP and leads to the cryptographic complexity of Cavefish. The LC and SP complete a blind signature protocol where the LC verifies if the transaction satisfies its defined specifications and only then creates valid signature(s) which are sent back to the SP. Finally, the SP attaches the signatures to the transaction and posts them on the blockchain on behalf of the LC.

To make our scheme viable in the real world, we adapt the blind predicate Schnorr signature scheme from [27]. More precisely, we do not require the unlinkability requirement once the transaction has been published on the blockchain. Accounting for timing, values and payees, the potential anonymity set for a blinded transaction is trivially small, and in any case the transaction signed by the LC only needs to stay *private until posted*. This insight allows us to introduce the notion of a *weakly* blind predicate signature scheme.

1.2 Contributions

Our light client protocol ensures that the SP is reimbursed for constructing the LC's transaction by including the payment in the transaction itself. For this reason, Cavefish does not require additional setup. In fact, our mechanism can be used to initialize a payment channel. Additionally, we show how we can improve both liveness and optimality of Cavefish with a small adjustment to the protocol to make it interact with multiple service providers.

To summarize, our contributions are:

- We introduce the notion of intents, describing the desired end result of the light client's actions, as opposed to the method by which they are accomplished. To this end, we describe a domain-specific language (DSL) allowing the concise construction of intents for UTxO-based ledgers.
- We propose Cavefish, a light client protocol with minimal set-up and chain state querying, which maintains safety, protects the SP from freeloading, and imposes minimal communication overhead. It is compatible with UTxO blockchains using Schnorr signatures or variants such as EdDSA [6]. We also describe how Cavefish can be made multi-SP aware for improved liveness.

- We introduce the notion of weakly blind predicate signatures, motivated by our “private until posted” goal. This brings together the notions of blind predicate signatures of [27] with the notion of signatures on committed messages (SBCM) [8].
- We implement and benchmark the zero-knowledge proof component of Cavefish for two major blockchain platforms, Bitcoin and Cardano, and show that an “unoptimized” implementation achieves proving and verification times that are viable in a real-world deployment.

2 Background and Related Work

2.1 Light Clients

Light clients are defined by the property that they interact with the blockchain without downloading (or fully validating) the full chain history. A “traditional” light client has the primary goal of obtaining some (either cryptographically guaranteed or trusted) data about the current state of the blockchain [2] [19]. Our LC indeed fits the definition of a light client, despite not following the traditional approach. Many realistic UTxO ledger implementations (e.g. Ergo [48], Cardano [11], and BitCoin [7]) present a unique challenge for monetizing LC operation. Such chains are set up in a way that an invalid transaction will not, in most cases, result in any update to the ledger state, but get rejected instead.

This positions services such as blockchain explorers [12] or APIs like Blockfrost [46] as convenient, low-cost (or free), and safe (transactions constructed with bad data are rejected) options on top of which to build traditional light clients. Light wallets such as VESPR [53] operate on this principle. VESPR queries Blockfrost, then constructs a transaction for the user which includes fees for using VESPR. A savvy user can use a hardware wallet such as Keystone to inspect a VESPR transaction in full [53], and remove any fee payments prior to submitting it. This vulnerability appears to not have yet been exploited for social rather than technical reasons.

In contrast, our LC design does not require revealing certain data sought by a user before their payment for this service appears on-chain. Fee payment and on-chain intent fulfillment are performed atomically by a single transaction with no preceding setup required. We believe this constitutes a sound monetization scheme for service providers to UTxO-chain light clients with a primary goal of submitting an intent-fulfilling transaction. Such a scheme could co-exist alongside the free Explorer or API service model, and potentially serve as a template for establishing universal monetization and decentralization of such services.

To verify the correctness of the data an SP provides, LC designs rely on either a single-prover or a multi-prover model. Single-prover light clients perform some cryptographic verification of the SP’s response. The Simplified Payment Verification (SPV) approach [42] is an example of single-prover light client design for UTxO-based ledgers. It requires the LC to download and verify block headers of the longest chain, but not the block bodies. The main cryptographic building blocks that are used to realize single-prover light clients are succinct representation and proofs, such as data accumulators (often Merkle trees), commitments and SNARKs (e.g. used to aggregate multi-signatures [52]). Examples of suitable cryptographic primitives for such designs include aggregate signatures and threshold signatures [2, 14].

Multi-prover light clients are able to trust the chain state supplied by a service provider if the data is confirmed by a sufficiently large set of full nodes [45, 47]. The design we present is a single-prover approach that can be naturally extended to a multi-prover setting based on a fingerprint provided by each prover, then choosing one to run the protocol based on the most popular fingerprint.

Some light clients use game-theoretic approaches by implementing the slashing of previously deposited collateral in case of misbehavior [39]. Unlike in the design we propose, such a payment scheme is not atomic for each transaction.

Another common technique is to compress the blockchain and/or ledger state to reduce the information that a light client requires to function reliably (see [10, 14, 28]). This, too, is different from our model, which does not require any chain data to be sent to the client.

2.2 Intents

Intent solver networks attempt to establish a relationship between solvers and users via their (light) clients [24]. A light client issues an intent as an abstracted transaction object. Then, solvers process the intent, incentivized via some intent execution reward. Ours appears to be the first work describing an intent-based light client protocol taking advantage of specific features of the UTxO ledger model that both minimizes communication and offers built-in incentive structure.

Cross-chain Intents A lot of work on intents is focused on building cross-chain intent resolution, which requires more sophisticated protocols that are able to communicate with multiple chains at once [58]. Some protocol designs choose to resolve intents on-chain directly [49], while others offer both layer-1 and layer-2 options [21]. Another design is a consensus protocol relying on minimal trust assumptions while conforming to the interledger standard [22]. Focusing on a single chain allows us to construct a protocol which makes use of the unique features of the underlying blockchain. In its simplicity and limited communication, our protocol also resembles payment channels.

2.3 Payment Channels

The concept of *payment channels*, and the related notion of *state channels*, also bear similarities to our approach. Like our light client, payment channels are a type of off-chain protocol for blockchains. A payment channel can be used to conduct a series of transactions without interacting with the main blockchain, e.g. [37] [50] [47]. The creation of a payment channel between parties requires locking funds via an on-chain transaction [40]. Some channels, such as Hydra [16], not only allow simple payments between users, but can simulate the majority of the on-chain transaction processing mechanism internally. This type of channel is usually called a *state channel*. State and payment channels require more pre-existing infrastructure and more per-user set-up than our design. Channels may also have a very limited set of possible actions, and require specifying sets of users as well as total liquidity at the time of initializing the channel. However, once initialized and entered into, they may be cheaper or faster than Cavefish, and equally reliable. Cavefish interacts well with payment channels: Cavefish can be used to setup a payment channel from a light client, and setting up a payment channel allows the SP to be compensated off-chain, thus avoiding the on-chain overhead of subsequent uses of Cavefish.

2.4 Hierarchical Deterministic Wallets and Address Discovery

UTxO based blockchains, including Bitcoin and Cardano use the concept of Hierarchical Deterministic (HD) wallets [57] where child addresses can be created in a deterministic way using a public key of a keypair sk, pk , and some short auxiliary data (the chain code). Given a public key pk , chain code c and requested index i for a child key, a hash function h is used to produce a scalar $s_i = h(c, i)$ enabling the derivation of the child key as $sk_i = sk + s_i$. It is also possible to use s_i without knowing sk to directly derive the corresponding public key $pk_i = pk \cdot g^{s_i}$. The same method is used to produce a child chain code c_i , so that multiple generations of children are possible.

In our application, we can assume that a wallet can give the service provider (SP) a single public key and chain code, letting the SP do address derivation and lookup on their side using some agreed-upon bounds or heuristics on the index depth. This keeps the communication cost between the light client and the SP constant.

2.5 Schnorr Blind Signatures

Schnorr signatures [43] have been replacing RSA in many scenarios due to their smaller size and faster verification. Compared to (EC)DSA, Schnorr offers similar efficiency and guarantees, but (EC)DSA security proofs are most likely not possible without strong idealization [33]. As a consequence, EdDSA [6], a popular variant of the Schnorr scheme, is currently under consideration by NIST for standardization.

Schnorr signatures are widely used in blockchains such as Bitcoin, Bitcoin Cash, Litecoin, and Polkadot. Monero, Zcash, and Cardano use the EdDSA variant. The adoption of Schnorr/EdDSA signatures is driven by privacy [9] and scalability gains [41] but also the extension to *blind Schnorr signatures* [20]. Most Schnorr-based blind signature schemes require multiple rounds [20], leading to denial-of-service susceptibility. Research has been focusing on making blind signatures *concurrently* secure where more than one session can be intertwined, such as in [4, 15, 32, 51].

Our construction hinges on the concurrently secure blind and partially blind signing protocol for standard Schnorr signatures [27] by Fuchsbaauer, et al.—the first work stating rigorous security guarantees for a practical blind signature scheme based on Schnorr signatures. Unlike schemes that had been presented before, [27] is not vulnerable to the attacks in [5, 55] which showed that the hardness-assumption¹ existing schemes relied on for concurrent security can be solved in polynomial-time. As our protocol is based on [27], it also provides partial and predicate blindness, two properties that allow us to describe a transaction in an abstract way featuring “redacted” parts. We outline the construction of Fuchsbaauer et al. [27] in the following. The scheme is equivalent to the blind signature scheme [20] by Chaum and Pedersen, but, to make the protocol concurrently secure, it adds a commitment phase at the beginning, where the user sends an encrypted version of the message m and blinding values (α, β) to the signer. Moreover, the second message from the user includes a zero-knowledge proof alongside the Schnorr challenge c . The zero-knowledge proof asserts to the signer that the initial (encrypted) commitment and c have been derived from the same m, α, β .

Besides obtaining concurrent security, one can leverage the zero-knowledge proof to assert additional facts, i.e., the user can prove to the signer that certain predicate(s) over message m hold. This turns the fully blind scheme into a predicate blind scheme. Furthermore, support for predicate blindness implies partial blindness [27] since the signer can construct a predicate that checks equality for part(s) of m .

3 Technical Background

Notation. We use $y \leftarrow x$ to denote that variable y is assigned the (possibly randomized) evaluation of x . When x is a set, we denote uniformly random sampling from the set. We use $a = b$ to denote boolean comparison between a and b and $z := w$ to denote equality by definition. We use $||$ to denote concatenation of bitstrings. When algorithms are randomized, we denote them as $A(x; r)$, where x is the input and r is the randomness, belonging to a randomness space $\mathcal{R}$. When we write $y \leftarrow A(x)$ we imply $r \leftarrow \mathcal{R}; y \leftarrow A(x; r)$. In algorithm descriptions, we write $x, y, z \subseteq a$ to imply parsing a to obtain x, y, z .

Our protocols operate in the discrete log setting, where we assume the discrete logarithm problem is hard. We also follow standard conventions with regards to public key encryption using $(\text{PKE.KeyGen}, \text{PKE.Enc}, \text{PKE.Dec})$ to describe schemes and require IND-CPA security. For signatures, we use Schnorr signatures $(\text{SDS.Setup}, \text{SDS.KeyGen}, \text{SDS.Sign}, \text{SDS.Ver})$. Finally, we parametrize the non interactive Arguments $(\text{NArg.Rel}, \text{NArg.Setup}, \text{NArg.Prove}, \text{NArg.Ver}, \text{NArg.SimProve})$ to allow the relation being proven to depend on the group setting.

¹The ROS (Random inhomogeneities in a Overdetermined Solvable system of linear equations) problem was initially studied by Schnorr in [44]

3.1 Discrete Log Groups

Definition 3.1. A group generator GGen is a probabilistic polynomial time (p.p.t.) algorithm with input a security parameter λ and outputs a group description $\mathbb{G}, g, q$ such that $\mathbb{G}$ is a group of prime order $q \approx 2^\lambda$ with generator g . We say that the discrete logarithm problem is hard w.r.t. GGen if for all p.p.t. $\mathcal{A}$ we have that

$$\Pr \left[(\mathbb{G}, g, q) \leftarrow \text{GGen}(1^\lambda); t \leftarrow \mathbb{Z}_q; h \leftarrow g^t : t = A(\mathbb{G}, g, q, h) \right] \text{ is negligible in } \lambda.$$

3.2 Pseudorandom Permutations

Definition 3.2 (Pseudorandom Permutation). Let $\lambda \in \mathbb{N}$ be a security parameter. A family of permutations

$$\mathcal{F} = \{F_k : \{0, 1\}^n \rightarrow \{0, 1\}^n\}_{k \in \{0, 1\}^\lambda}$$

is called a *pseudorandom permutation* (PRP) if for every probabilistic polynomial-time (PPT) adversary $\mathcal{A}$,

$$\left| \Pr_{k \leftarrow \{0, 1\}^\lambda} \left[\mathcal{A}^{F_k, F_k^{-1}}(1^\lambda) = 1 \right] - \Pr_{\pi \leftarrow \text{Perm}(n)} \left[\mathcal{A}^{\pi, \pi^{-1}}(1^\lambda) = 1 \right] \right|$$

is negligible in λ , where $\text{Perm}(n)$ denotes the set of all permutations over $\{0, 1\}^n$, and $\mathcal{A}$ is given oracle access to the permutation and its inverse.

3.3 Public Key Encryption

A public key encryption scheme PKE comprises a set of polynomial time algorithms KeyGen , Enc , Dec with the following syntax:

- $\text{KeyGen}(1^\lambda) \rightarrow (ek, dk)$. Creates a public/private keypair ek, dk . The encryption key ek also defines the message space $\mathcal{M}$
- $\text{Enc}(ek, m; \rho) \rightarrow C$. Encrypts a message $m \in \mathcal{M}$ under the public encryption key ek , using encryption randomness ρ . We specify ρ explicitly when we wish to refer to it later, and omit it when it is not needed.
- $\text{Dec}(dk, C) \rightarrow m$. Decrypts a ciphertext C using the private decryption key dk .

Definition 3.3. A public key encryption scheme PKE is *correct* if

$$\Pr[(ek, dk) \leftarrow \text{PKE.KeyGen}(1^\lambda); m \leftarrow \mathcal{M} : \text{PKE.Dec}(dk, \text{PKE.Enc}(ek, m)) = m] = 1.$$

Definition 3.4. A public key encryption scheme PKE is *IND-CPA secure* if for all stateful p.p.t. adversaries $\mathcal{A}$, the difference

$$\left| \Pr \left[(ek, dk) \leftarrow \text{PKE.KeyGen}(1^\lambda); (m_0, m_1) \leftarrow A(ek); d \leftarrow \{0, 1\}; c^* \leftarrow \text{PKE.Enc}(ek, m_d) : A(c^*) = d \right] - \frac{1}{2} \right|$$

is negligible in λ .

3.4 Digital Signatures and Schnorr Signatures

Definition 3.5. The Schnorr digital signature scheme SDS is defined for a group generator GGen and a hash function generator HGen and operates as shown on Figure 1.

Definition 3.6. A digital signature scheme DS is *correct* if

$$\Pr \left[sp \leftarrow \text{DS.Setup}(1^\lambda); (sk, vk) \leftarrow \text{DS.KeyGen}(sp); m \leftarrow \mathcal{M}_S : \text{DS.Ver}(vk, m, \text{Sign}(sk, m)) = 1 \right] = 1.$$

<u>SDS.Setup(1^λ)</u> $(\mathbb{G}, g, q) \leftarrow \text{GGen}(1^\lambda)$ $H \leftarrow \text{HGen}(q)$ $sp \leftarrow (\mathbb{G}, g, q, H)$ return sp	<u>SDS.KeyGen(sp)</u> $(\mathbb{G}, g, q) : \subseteq sp$ $x \leftarrow \mathbb{Z}_q; X \leftarrow g^x$ $sk, vk \leftarrow (par, x), (par, X)$ return (sk, vk)
<u>SDS.Sign(sk, m)</u> $(\mathbb{G}, g, q, H, x) : \subseteq sk$ $r \leftarrow \mathbb{Z}_q; R \leftarrow g^x; c \leftarrow H(R, X, m)$ $s \leftarrow (r + c \cdot x) \bmod q$ return $\sigma \leftarrow (R, s)$	<u>SDS.Ver(vk, m, σ)</u> $(\mathbb{G}, g, q, H, X) : \subseteq vk$ $(R, s) \leftarrow \sigma; c \leftarrow H(R, X, m)$ return $(g^s = R \cdot X^c)$

Fig. 1. The Schnorr signature scheme SDS, with group and hash generators GGen, HGen.

Definition 3.7. A digital signature scheme DS is *strongly existentially unforgeable against chosen message attacks* (sEUF-CMA) if for all p.p.t. adversaries $\mathcal{A}$, we have

$$\Pr[\text{sEUF-CMA}_{\text{DS}}^{\mathcal{A}}(1^\lambda)]$$

is negligible in λ where the game $\text{GsEUF-CMA}_{\text{DS}}^{\mathcal{A}}$ is defined in Figure 2.

<u>GsEUF-CMA$_{\text{DS}}^{\mathcal{A}}(1^\lambda)$</u> $sp \leftarrow \text{DS.Setup}(1^\lambda); Q \leftarrow \emptyset$ $(sk, vk) \leftarrow \text{DS.KeyGen}(sp)$ $(m^*, \sigma^*) \leftarrow \mathcal{A}^{\text{OSig}}(vk)$ return $(m^*, \sigma^*) \neq Q \wedge \text{DS.Ver}(vk, m^*, \sigma^*)$	<u>OSig(m)</u> $\sigma \leftarrow \text{DS.Sign}(sk, m)$ $Q \leftarrow Q \cup \{(m, s)\}$ return σ
---	---

Fig. 2. The security experiment $\text{sEUF-CMA}_{\text{DS}}^{\mathcal{A}}$ and supporting oracle OSig.

The security of Schnorr signatures has been well studied in the random oracle model (ROM), where the hash is replaced with an ideal random function the adversary can only call as an oracle. However, in this work, the need arises to instantiate the underlying hash function H so that we may reason about it within a proof system [3], e.g., prove that a known y is $y = H(x)$ for some secret x . This renders ROM-based proofs inapplicable necessitating an assumption on the security of the scheme. This does not differ significantly from typical implementation practices (where the hash function is in fact drawn from a small pool for standardized options), and also from the treatment of this issue in the literature [27].

Definition 3.8. A hash function generator $\text{HGen}(n) \rightarrow H$, on input $n \in \mathbb{N}$ generates a hash function $H : \{0, 1\}^* \rightarrow \mathbb{Z}_n$.

Assumption 1. [27] There exists a group generator GGen and hash function generator HGen such that the Schnorr signature scheme SDS is strongly unforgeable under definition 3.7.

Similarly, in the ROM we can trivially guarantee H is pseudorandom, whereas for our setting we require an additional assumption:

Assumption 2. The hash function generator HGen is such that for all n

$$\left| \Pr[H \leftarrow \text{HGen}(n); (m_0, m_1) \leftarrow \mathcal{A}(H); b \leftarrow \{0, 1\}; r \leftarrow \{0, 1\}^\lambda : b = \mathcal{A}(H(m_b || r))] - 1/2 \right|$$

is negligible in λ .

3.5 Parametrized Non-Interactive Zero Knowledge Arguments

Following [27], we define non-interactive zero knowledge arguments with regards to parametrized polynomial relations $\mathcal{P} : \{0, 1\}^* \times \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}$, where the first argument represents a parameter set par (e.g. a group description). Given a value of par , we say that w is a witness for statement θ if $\mathcal{P}(par, \theta, w) = 1$, i.e. $R = R_{par}(\theta, w) := \mathcal{P}(par, \theta, w)$ is an NP-relation, and $\mathcal{L} = \mathcal{L}_{par}$ is an NP-language. A NIZK for a relation $\mathcal{P}$ operates as follows:

- $\text{Rel}(1^\lambda) \rightarrow par$. Generates a parameter set par that defines R and $\mathcal{L}$.
- $\text{Setup}(par) \rightarrow (\text{crs}, \tau)$. Generates a common reference string (CRS) and trapdoor τ used by the simulator. We assume the CRS contains a description of R .
- $\text{Prove}(\text{crs}, \theta, w) \rightarrow \pi$. Given a CRS crs , statement θ and witness w for θ , produces a proof π .
- $\text{Ver}(\text{crs}, \theta, \pi) \rightarrow \{0, 1\}$. Given a CRS crs , a statement θ and a proof π outputs 1 or 0, accepting or rejecting the proof.
- $\text{SimProve}(\text{crs}, \theta, \tau)$. Given a CRS crs , statement θ and trapdoor τ for crs , produces a simulated proof π .

Definition 3.9. A system $\text{NArg}[R]$ is perfectly correct if for all unbounded adversaries $\mathcal{A}$

$$\Pr[par \leftarrow \text{NArg.Rel}(1^\lambda); (\text{crs}, \tau) \leftarrow \text{NArg.Setup}(par); (\theta, w) \leftarrow A(\text{crs}) : \neg R(\theta, w) \vee \text{NArg.Ver}(\text{crs}, \theta, \text{NArg.Prove}(\text{crs}, \theta, w))] = 1$$

Definition 3.10. A system $\text{NArg}[R]$ is adaptably computationally sound if for all p.p.t. adversaries $\mathcal{A}$

$$\Pr[par \leftarrow \text{NArg.Rel}(1^\lambda); (\text{crs}, \tau) \leftarrow \text{NArg.Setup}(par); (\theta, \pi) \leftarrow A(\text{crs}) : \neg \mathcal{L}(\theta) \wedge \text{NArg.Ver}(\text{crs}, \theta, \pi)]$$

is negligible in λ .

Definition 3.11. A system $\text{NArg}[R]$ is computationally zero-knowledge if for all p.p.t. adversaries $\mathcal{A}$

$$\left| \Pr[par \leftarrow \text{NArg.Rel}(1^\lambda); (\text{crs}, \tau) \leftarrow \text{NArg.Setup}(par); d \leftarrow \{0, 1\} : d = A^{\text{OProve}_d}(\text{crs})] - \frac{1}{2} \right|$$

is negligible in λ , where

$$\begin{aligned} \text{OProve}_0(\theta, w) &:= \text{if } \neg R(\theta, w) \text{ return } \perp; \text{ return } \text{NArg.Prove}(\theta, w), \\ \text{OProve}_1(\theta, w) &:= \text{if } \neg R(\theta, w) \text{ return } \perp; \text{ return } \text{NArg.SimProve}(\theta, \tau). \end{aligned}$$

3.6 Ledger Model

The ledger model to which we tailor our light client design is (a slightly pared down version of) the extended UTxO ledger with multi-asset support (EUTxO_{ma}), first introduced in [18]. The UTxO ledger model, such as the one used by BitCoin [42], Ergo [26], and Cardano [13], maintains a record, called the *UTxO set*, of transaction outputs added by transactions that have been applied throughout its history, but not yet spent by subsequent transactions.

The extended UTxO ledger with multi-asset support allows users to transact with multiple kinds of assets, as well as include user-defined scripts to be run during transaction processing. We work with a simplified, non-Turing-complete,

$\mathbb{H} = \bigcup_{n=0}^{\infty} \{0, 1\}^{8n}$	the type of bytestrings
$(a, b) : \text{Interval}[A]$	intervals over a totally-ordered set A
$[a1; \dots; ak] : [C]$	finite list with terms of type C
$h :: t : [C]$	list with head h and tail t
$x \cup \text{nothing} : A^?$	maybe type over A
$a \{ \text{field} = x \} : A$	record of type A with field changed to x
$\text{Key} \mapsto \text{Value} \subseteq$	$\{ k \mapsto v \mid k \in \text{Key}, v \in \text{Value} \}$
	finite map with unique keys

Fig. 3. Notation

version of the EUTxO_{ma} ledger. That is, instead of allowing arbitrary user-defined scripts to be run, we constrain the possible scripts to only those composed using a small DSL we provide here. In the interest of keeping this DSL small, we omit support for some user inputs to script validation, called *redeemers*, found in the full model. These changes simplify the examples we present while still allowing us to demonstrate relevant usecases of our light client design.

The basic UTxO-style ledger transaction structure is as follows (note that realistic ledgers often contain additional data in both the transaction and the UTxO set):

$$\begin{aligned} \text{Tx} = & (\text{inputs} : \text{Set TxIn}, \text{outputs} : [\text{TxOut}], \\ & \text{validityInterval} : \text{Interval}[\text{Slot}], \\ & \text{mint} : \text{Value}, \text{fee} : \mathbb{N}, \text{aux} : \mathbb{H}, \text{sigs} : \text{Signature}) \end{aligned}$$

The UTxO set is a finite map, $\text{UTxO} := \text{TxIn} \mapsto \text{TxOut}$. A transaction updates the UTxO set by adding and/or removing entries. The EUTxO_{ma} ledger model is well-established, discussed in previous work, e.g. [18, 54], and we give an overview of the model for completeness below as well.

EUTxO Model Overview. In Figure 3, we introduce the standard ledger syntax that we use throughout.

Blocks and Ledger States. A block $\text{Block} = \text{Header} \times [\text{Tx}]$ is a data structure used to update the state of the ledger by applying a list of transactions $\text{lstx} \in [\text{Tx}]$ contained in the block, as well as doing some other checks and updates we do not model here. Among other data, the block header contains a slot number field $\text{slot} : \text{Header} \rightarrow \text{Slot}$, which represents the blockchain time at which the block is produced.

The ledger state is a data structure which is updated by applying blocks incoming on the network. The ledger state LState contains (among other data we do not model here) the UTxO set field, $\text{utxo} : \text{LState} \rightarrow \text{UTxO}$. It also contains the parameter $\text{minfee} : \text{LState} \in \mathbb{N}$, which is the minimum fee a transaction must pay. A block b can extend the blockchain (i.e. update the current state $s \in \text{LState}$) whenever the function

$$\text{checkBlock} : \text{LState} \times \text{Block} \rightarrow \{0, 1\}$$

applied as $\text{checkBlock}(s, b)$ returns 1. Then, the updated block state is computed by $\text{updateState}(s, b)$. We do not give full specifications of checkBlock or updateState , as they are not required to model our light client approach.

Full Nodes. Let s_0 be some verified state, e.g. a genesis state, or a verified checkpoint state, and suppose $[b_0, \dots, b_k]$ is a list of blocks that have been disseminated across the network since the time slot of s_0 . We assume that a *full node* is one that is able to (1) compute the current state s_k by applying them in sequence, i.e. computing $s_{i+1} = \text{updateState}(s_i, b_i)$ for $0 \leq i < k$, (2) is able to determine if all of the blocks in the list are valid (i.e. $\text{checkBlock}(s_i, b_i) = 1$), and (3)

can produce, send, and receive blocks on the network. If all the blocks are valid, we say that $[b_1, \dots, b_k]$ forms a valid blockchain.

UTxO Structure. An input $(txid, ix) \in \text{TxIn} := \text{TxId} \times \mathbb{N}$ is a pair of a transaction ID and a natural number. When a transaction is applied to a UTxO set, its set of inputs is used to identify the entries which the transaction is removing from the set. In each input $i = (txid, ix)$, $txid \in \text{TxId} := \mathbb{H}$ is the hash of a (previous) transaction that added that entry to the UTxO, and ix is the index of that output in the list of outputs of that transaction. An output $(s, v, d) \in \text{TxOut} := \text{Script} \times \text{Value} \times \text{Data}$ is a tuple of a script s that is made up of the following three components :

(i) *Script.* $s \in \text{Script}$ specifies constraints on a transaction, such as requiring it to be signed by a specific key, or pay to a specific address. The Script constructors and evaluation function $\llbracket _ \rrbracket$ are discussed in more detail in Section 3.7. This DSL is used in our ledger model instead of working with arbitrary scripts. In an EUTxO model, script evaluation is not a state-updating computation, it merely outputs 0 or 1 depending on whether the transaction satisfies its constraints. Transaction validation fails if any of its scripts fail to validate (return 0).

(ii) *Value.* Our ledger model has native *multi-asset support*, following the scheme outlined in prior work [17]. That is, the $v \in \text{Value}$ in the output contains not only a quantity of a single currency (like BitCoin), but an arbitrary finite number of different types of assets with minting policies and unique identifiers alongside their quantities.

(iii) *Data.* The bytestring $d \in \text{Data}$ in an output is specified by the user at the time of transaction construction. In general, it can be arbitrary. However, it is often used to represent the state of a stateful smart contract, and is therefore constrained in accordance with the specification of that contract. Stateful contracts in the EUTxO_{ma} model are implemented using a conjunction of output-locking and minting Scripts [18].

The list outputs of outputs of a transaction tx is used to construct a set of UTxO entries that will be added to the UTxO set, such that the unique identifier $txin$ of each output $o \in \text{outputs } tx$ consists of the transaction hash $txid$ tx , and the index of o in the list $\text{outputs } tx$. The entries added to the UTxO set by tx are computed in this way by mkOuts , see Figure 5.

The interval validityInterval specifies the range of slot numbers for which a transaction can be valid, or nothing whenever there is no limitation. The field $\text{signs} : \text{Signature} := \text{PubKey} \mapsto \mathbb{H}$ is a set of public keys, associated with their signatures on the the transaction (excluding signs itself). The fee is the amount of primary currency a transaction pays as a system fee, which is must be at least the required fee minfee .

The Value in the mint field specifies the assets being minted or burned by the transaction. Assets with positive quantities are said to be minted, while those with negative quantities are burned. When a transaction is applied, the constraints specified by every $p \in \text{Policy} := \text{Script}$ of each type of asset specified in this field are checked to make sure minting/burning of this type and quantity of asset is allowed. The aux is a field for arbitrary extra data encoded as a bytestring, e.g. a "note".

Ledger State Update. Given a UTxO set $utxo$ and a transaction tx , the function $\text{updateUTxO} : \text{UTxO} \times \text{Tx} \rightarrow \text{UTxO}$ computes the updated UTxO set by adding and removing the appropriate entries:

$$\text{updateUTxO}(utxo, tx) = \{ i \mapsto o \in utxo \mid i \notin \text{inputs}(tx) \} \cup \text{mkOuts}(tx)$$

While an update to the UTxO set can be computed for any transaction, only transactions that are *valid* for a given set are allowed to perform an update to the ledger state. For a given $utxo$ set, a transaction tx is valid whenever the function $\text{checkTx} : (\text{Slot} \times \mathbb{N}) \times \text{UTxO} \times \text{Tx} \rightarrow \{0, 1\}$, applied as $\text{checkTx}((\text{slot}, \text{fee}), utxo, tx)$, returns 1. The checkTx

function is the conjunction of the constraints specified by the following validation rules: a transaction tx is *valid* if it follows rules (i) to (xi) below.

(i) **The transaction has at least one input:**

$$tx.inputs \neq \{\}$$

(ii) **The current slot is within transaction validity interval:**

$$slot \in tx.validityInterval$$

(iii) **All outputs have positive values:**

$$\forall o \in tx.outputs, o.value > 0$$

(iv) **All output references of transaction inputs exist in the UTxO:**

$$inputs\ tx \subseteq \text{dom } utxo$$

(v) **Value is preserved:**

$$tx.mint + \sum_{\substack{i \in tx.inputs, \\ (i \mapsto o) \in utxo}} o.value = \sum_{o \in tx.outputs} o.value + \text{toValue}(tx.fee)$$

(vii) **All inputs validate:**

$$\forall i \in tx.inputs, i \mapsto (s, v) \in utxo, \llbracket s \rrbracket(tx) = 1$$

(ix) **All minting scripts validate:**

$$\forall p \mapsto _ \in tx.mint, \llbracket p \rrbracket(tx) = 1$$

(x) **All signatures are correct:**

$$\forall (pk \mapsto s) \in tx.sigs, \text{checkSig}(tx.txid, pk, s) = 1$$

(xi) **The fee is sufficient:**

$$s.minfee \leq tx.fee$$

3.7 Script construction and Evaluation

In Figure 4, we present the constructors and evaluation rules for scripts, and in Figure 11 we explain the transaction building function `mkToSpec`.

For a given block b and state s , the function `updateState` (s, b) updates s with the list of transactions $\pi_2 b = [tx_1; \dots; tx_k]$ in such a way that the update to the UTxO set contained in s is computed by applying the transactions in sequence, i.e.

$$\text{updateUTxO}(\text{updateUTxO}(\dots \text{utxo } s \dots), tx_{k-1}), tx_k) = \text{utxo}(\text{updateState}(s, b))$$

For each transaction in the list, `checkTx` ((slot b , fee s), $utxo_i$, tx_i) = 1 is first checked, and the entire block containing it is considered invalid if this check fails.

CONSTRUCTORS OF Script			EVALUATION OF Script	
RequireMOf	$: \mathbb{N} \rightarrow [\text{Script}] \rightarrow \text{Script}$		$\llbracket _ \rrbracket : \text{Script} \rightarrow \text{Tx} \rightarrow \{0, 1\}$	
RequireSig	$: \text{PubKey} \rightarrow \text{Script}$		$\llbracket \text{RequireMOf } n \text{ } ls \rrbracket (tx) = \text{MOf } 0 \text{ } n \text{ } (\llbracket _ \rrbracket (tx)) \text{ } ls$	
RequireTimeStart	$: \text{Slot} \rightarrow \text{Script}$		$\llbracket \text{RequireSig } k \rrbracket (tx) = k \in khs$	
RequireTimeExpire	$: \text{Slot} \rightarrow \text{Script}$		$\llbracket \text{RequireTimeStart } t1' \rrbracket (tx) = t1' \leq^? t1$	
NextStep	$: \text{Output} \rightarrow \text{Script}$		$\llbracket \text{RequireTimeExpire } t2' \rrbracket (tx) = t2 \leq^? t2'$	
			$\llbracket \text{NextStep } (s, v, d) \rrbracket (tx) = (s, v, d) \in \text{outputs}$	
			where	
			$khs := \text{dom } (tx.\text{sigs})$	
			$t1 := \text{proj}_1 (tx.\text{validityInterval})$	
			$t2 := \text{proj}_2 (tx.\text{validityInterval})$	

Fig. 4. Script constructors and evaluation

LEDGER PRIMITIVES	
$\text{checkSig} : \mathbb{H} \rightarrow \text{PubKey} \rightarrow \mathbb{H} \rightarrow \{0, 1\}$	
<i>checks that a given key signed (the hash of) a transaction</i>	
HELPER FUNCTIONS	
$\text{txid} : \text{Tx} \rightarrow \text{TxD}$	
$\text{txid } tx = \text{hash } (tx \{ \text{sigKeys} = \text{dom } (tx.\text{sigs}), \text{sigs} = \emptyset \})$	
$\text{toMap} : \mathbb{N} \rightarrow [\text{TxOut}] \rightarrow (\mathbb{N} \mapsto \text{TxOut})$	
$\text{toMap}(_, []) = []$	
$\text{toMap}(ix, u :: \text{outs}) = \{ ix \mapsto u \} \cup \text{toMap}(ix + 1, \text{outs})$	
<i>constructs a map from a list of outputs</i>	
$\text{mkOuts} : \text{Tx} \rightarrow \text{UTxO}$	
$\text{mkOuts}(tx) = \{ (tx, ix) \mapsto o \mid (ix \mapsto o) \in \text{toMap}(0, tx.\text{outputs}) \}$	
<i>constructs a UTxO set from a list of outputs of a given transaction</i>	
$\text{MOf} : \mathbb{N} \rightarrow \mathbb{N} \rightarrow (A \rightarrow \{0, 1\}) \rightarrow [A] \rightarrow \{0, 1\}$	
$\text{MOf } k \text{ } m \text{ } f \text{ } [] = m \leq k$	
$\text{MOf } k \text{ } m \text{ } f \text{ } (h :: t) = \text{if } (m \leq k) \text{ then } 1 \text{ else } (\text{MOf } (k + a) \text{ } m \text{ } f \text{ } t)$	
where $a = \text{if } (f(h)) \text{ then } 1 \text{ else } 0$	
<i>returns 1 if enough elements of a list satisfy given function</i>	

Fig. 5. Primitives and basic types for the UTxO_{ma} model

4 Light Client and Intent Specification

The primary goal of a light client is to construct an intent to be fulfilled by an SP. The light client specifies its intent using a domain-specific language (DSL) we define here. The SP responds with an *abstract transaction*, which partially conceals data in a way that (i) allows the light client to verify that their intent has been resolved in the original transaction and (ii) is not sufficient to construct a valid transaction that avoids paying the service provider fee.

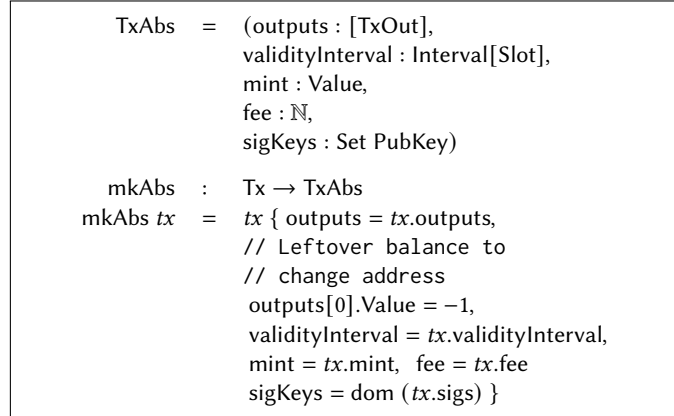
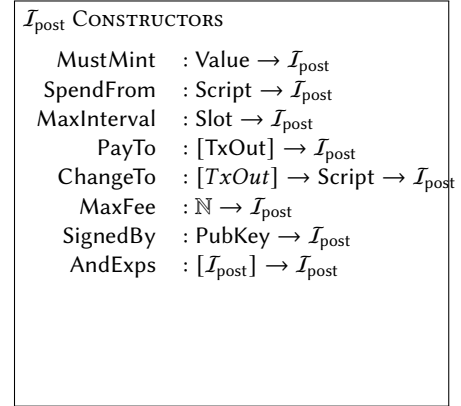


Fig. 6. Abstract transaction type TxAbs and constructor mkAbs

Fig. 7. $\mathcal{I}_{\text{post}}$ constructors and evaluation

A light client's constraints may be on the communication cost to answer a series of queries, available state/data storage, and assumptions about connectivity. This limits the set of intents it can construct. In this work, we assume the intents are limited only by the power of the DSL we use. The design of the DSL is minimal, and is meant only to showcase the operation of the light client protocol. To expand the functionality of the light client, the DSL can be swapped for a Turing-complete script-writing language.

An *abstract transaction*, TxAbs, is the data structure that a light client receives from the service provider instead of the full plaintext transaction (see Fig. 6). In TxAbs, the aux field and the inputs fields are removed, and instead of the Signature field, it contains only the signing keys sigKeys and not the corresponding signatures. The value of the first output is zeroed out to allow for any leftover input funds to be returned to the LC's change address. The intent a light client constructs is any expression in the DSL $\mathcal{I}_{\text{post}}$, see Figure 7.

For the implementation of the example use cases of our design, we require the definition of the following functions:

- (i) The function $\text{mkToSpec} : \text{LState} \times \mathcal{I}_{\text{post}} \rightarrow \text{Tx}^?$ that constructs a transaction based on the specification (this function may fail, outputting nothing instead of a transaction, if the intent cannot be resolved), see Fig. 11, 10,9.
- (ii) The function $\text{chkSpec} (\mathcal{I}_{\text{post}} \times \text{TxAbs}) \rightarrow \{0, 1\}$ that checks that a given abstract transaction matches the intent specification, see Fig. 7, and Fig. 8.

The function mkToSpec constructs a transaction by editing an initial transaction $\text{initTx}_{a,mf}$ (see Figures 11,10,9). This transaction includes a minimum fee mf as well as a tip to the service provider, which is output to its address a . These functions are defined such that for any l, i , where $\text{mkToSpec} (l, i) \neq \text{nothing}$, necessarily $\llbracket i \rrbracket_{\text{DSL}} (\text{mkAbs} (\text{mkToSpec} (l, i))) = 1$.

We present two examples for LC intents. The resulting transactions are given in Figure 12, assuming the intents can be resolved using the contents of the UTxO at the time of transaction construction. Given a given ledger state l , we define:

- (tx₁) Intent to mint some token t and store it at address a within an interval of length j and maximum fee $f \leq \text{minfee } l$:
 $i_1 = \text{AndExps} [\text{MustMint } t; \text{MaxInterval } j; \text{MaxFee } f; \text{SpendFrom} (\text{RequireSig } k);$
 $\text{ChangeTo} [(t, a, \text{" "})] s].$ This intent allows the SP to pick any outputs at script address $\text{RequireSig } k$ to cover the tip.

$$\begin{aligned}
\llbracket _ \rrbracket_{\text{DSL}} &: \mathcal{I}_{\text{post}} \rightarrow \text{TxAbs} \rightarrow \{0, 1\} \\
\llbracket \text{MustMint } v \rrbracket_{\text{DSL}}(tx) &= v \leq tx.\text{mint} \\
\llbracket \text{SpendFrom } s \rrbracket_{\text{DSL}}(tx) &= \llbracket s \rrbracket tx \\
\llbracket \text{MaxInterval } i \rrbracket_{\text{DSL}}(tx) &= (tx.\text{validityInterval})_2 - \\
&\quad (tx.\text{validityInterval})_1 \leq i \\
\llbracket \text{PayTo } o_1 \dots o_q \rrbracket_{\text{DSL}}(tx) &= o_1 \dots o_q \in tx.\text{outputs} \\
\llbracket \text{ChangeTo } p \rrbracket_{\text{DSL}}(tx) &= (s, _) = p \cap tx.\text{outputs} \\
\llbracket \text{MaxFee } f \rrbracket_{\text{DSL}}(tx) &= tx.\text{fee} \leq f \\
\llbracket \text{AndExps } [a_1; \dots; a_k] \rrbracket_{\text{DSL}}(tx) &= (\llbracket a_1 \rrbracket_{\text{DSL}} tx) \wedge \dots \\
&\quad \wedge \dots \wedge (\llbracket a_k \rrbracket_{\text{DSL}} tx)
\end{aligned}$$

Fig. 8. Evaluation of $\mathcal{I}_{\text{post}}$

TRIVIAL TRANSACTION $\text{initTx}_{a,mf}$

$$\begin{aligned}
\text{initTx}_{a,mf} &= \{ \text{inputs} = \emptyset, \\
&\quad \text{outputs} = [(a, \text{tip}, \text{"})], \\
&\quad \text{validityInterval} = \\
&\quad [\text{nothing}, \text{nothing}], \\
&\quad \text{mint} = 0, \\
&\quad \text{fee} = mf \\
&\quad \text{aux} = [] \\
&\quad \text{sigs} = \emptyset \}
\end{aligned}$$

Fig. 9. Trivial transaction function for transaction construction

INPUT SELECTION FUNCTION

$$\begin{aligned}
\text{mkIns} &: (\text{LState} \times (\text{Set}(\text{TxIn} \times \text{Data})) \times \text{Value} \times \text{Script}) \rightarrow (\text{Set}(\text{TxIn} \times \text{Data})) \\
\text{mkIns}(l, i, v, s) &= \text{if } \neg (v \leq 0) \text{ then} \\
&\quad \text{if } (v > 0) \text{ then} \\
&\quad \quad \text{funmkIns}(l \setminus (\{j \mapsto _ \}), i \cup j, v - (u(j).\text{value}), s) \\
&\quad \quad \text{else } (i, \text{"}) \\
&\quad \text{else} \\
&\quad \quad \text{nothing} \\
&\quad \text{where} \\
&\quad \quad j = (\text{pickInput } l \ s, \text{"}) \\
&\quad \quad u = \text{utxo } l
\end{aligned}$$

Fig. 10. Input selection function for transaction construction

(tx_{i2}) Intent to interact with a specific smart contract (one with the unique token identifier t and script s), paying value p into it from script $\text{RequireSig } k_1$, and updating its state to d : $i_1 = \text{AndExps } [\text{SpendFrom } (\text{RequireSig } k_1); \text{PayTo}[(s, t + p, d)]; \text{ChangeTo } [(s, t + p, d)] (\text{RequireSig } k_1)]$

The minimum required capacity of our light client is to perform all the actions that are specified in the protocol we describe in the next section.

Sub-optimality. In Section 7, we discuss the possibility of SPs supplying transactions that satisfy the client's intent albeit in a sub-optimal fashion, and we describe how multiple SPs can be leveraged to satisfy intents more efficiently.

5 Light Client Protocols

We give an overview on our light client protocol and its components. Our protocol maximizes space and communication efficiency by essentially pushing all storage and lookup tasks to the SP, whilst guaranteeing the resulting transaction is safe for the light client to sign, and compensating their SP for their effort. This is non-trivial as the transaction must be signed sight-unseen (blinded) otherwise, the light client could simply remove the payment to the SP and submit the modified transaction.

To satisfy both parties, we rely on cryptographic tools as well as some established features in implemented blockchains such as transaction validity periods and handling of invalid transactions. In the rest of this section, we make the following

AUXILIARY mkToSpec' DEFINITION

$$\begin{aligned}
& \text{mkToSpec}' : \text{LState} \rightarrow \mathcal{I}_{\text{post}} \rightarrow \text{Tx} \rightarrow \text{Tx}' \\
& \text{mkToSpec}' \ l \ (\text{MustMint } v)(tx) = tx \ \{ \text{mint} = v + tx.\text{mint}, \\
& \quad \text{sigs} = tx.\text{sigs} \cup \text{getSigsVal } v, \\
& \quad \text{validityInterval} = \text{restrictIntervalVal } tx.\text{validityInterval } v \} \\
& \text{mkToSpec}' \ l \ (\text{SpendFrom } s)(tx) = tx \ \{ \text{inputs} = tx.\text{inputs} \\
& \quad \cup \text{newIns}, \text{sigs} = tx.\text{sigs} \cup \text{getSigsUTxO newIns } l, \\
& \quad \text{validityInterval} = \text{restrictIntervalUTxO } tx.\text{validityInterval newIns } l \} \\
& \quad \textbf{where} \\
& \quad \text{newIns} = \text{mkIns } l \ (tx.\text{inputs}) \ (\text{produced} - \text{consumed}) \ s \\
& \text{mkToSpec}' \ l \ (\text{MaxInterval } i)(tx) = tx \ \{ \text{validityInterval} = (l.\text{slot}, \\
& \quad \min \{ l.\text{slot} + i, tx.\text{validityInterval}_2 \}) \} \\
& \text{mkToSpec}' \ l \ (\text{PayTo } [(s, v, d)])(tx) \\
& \quad = tx \ \{ \text{outputs} = (s, v, d) : (tx.\text{outputs}) \} \\
& \text{mkToSpec}' \ l \ (\text{ChangeTo } p \ s)(tx) = \text{if } v > 0 \\
& \quad \text{then } tx \ \{ \text{outputs} = (s, v, "") : (tx.\text{outputs}) \} \\
& \quad \textbf{where } v = \text{sumVals } (tx.\text{outputs}) - (\text{sumVals } p) \\
& \quad \text{else nothing} \\
& \text{mkToSpec}' \ l \ (\text{MaxFee } f) \ (tx) = \text{if } tx.\text{fee} \leq f \text{ then } tx \text{ else nothing} \\
& \text{mkToSpec}' \ l \ (\text{AndExps } [a1; a2; \dots; ak])(tx) = \text{mkToSpec}' \ l \ a_k \\
& \quad (\dots (\text{mkToSpec}' \ l \ a_2 (\text{mkToSpec}' \ l \ a_1 tx)))
\end{aligned}$$

mkToSpec DEFINITION

$$\begin{aligned}
& \text{mkToSpec} : (\text{LState} \times \mathcal{I}_{\text{post}}) \rightarrow \text{Tx}' \\
& \text{mkToSpec} \ (l, i) = \text{mkToSpec}' \ l \ i \ \text{initTx}_{a, mf}
\end{aligned}$$

Fig. 11. Building transactions according to the specific intent

$ \begin{aligned} \text{txi}_1 = \{ & \text{inputs} = \{ \text{inp}_1, \text{inp}_2 \}, \\ & \text{outputs} = [(s, t, "")], \\ & \text{validityInterval} = [\text{slot } l, (\text{slot } l) + j], \\ & \text{mint} = t, \\ & \text{fee} = \text{minfee } l \\ & \text{sigKeys} = \text{getSignersVal } t \} \end{aligned} $	$ \begin{aligned} \text{txi}_2 = \{ & \text{inputs} = \{ \text{inp}_1, \text{inp}_2, \text{inp}_3 \}, \\ & \text{outputs} = [(s, t + p, d), \\ & \quad (\text{RequireSig } k1, "")], \\ & \text{validityInterval} = [\text{nothing}, \text{nothing}], \\ & \text{mint} = \{ \}, \\ & \text{fee} = \text{minfee } l \\ & \text{sigKeys} = \{ k1 \} \} \\ & \textbf{where} \\ & \text{out}_1.\text{value} + \text{out}_2.\text{value} + \text{out}_3.\text{value} \\ & \quad - t - p - \text{minfee} - \text{tip} \end{aligned} $
--	---

Fig. 12. Abstract transaction examples

assumption: all communication between a light client and a service provider happens over a secure channel (e.g. a TLS connection).

5.1 Definitions and Requirements

Definition 5.1 (Correctness). A transaction building protocol is **correct** if for an honest light client, and service provider and any state $LState_i \in LState$ and intent $int_{post} \in I_{post}$ such that: if $mkToSpec(LState_i, int_{post}) \neq \perp$, then the protocol completes and transaction Tx' produced by the SP is s.t. $checkTxL(LState_i, Tx') = 1$.

We use $checkTxL(l, tx)$ as shorthand for $checkTx((slot, fee), utxo, tx)$, with ledger state l containing UtxO state $utxo$ and current slot number $slot$ and minimum fee fee .

Definition 5.2 (Safety). A transaction building protocol is **safe** if for an honest light client and any service provider SP we have that: for any transaction Tx' is produced by the SP is s.t. $checkTxL(LState, Tx') = 1$, then it must be that $chkSpec(int_{post}, Tx') = 1$.

The safety property is derived from two factors: first, the unforgeability property of $WBPS[P]$ will ensure that only transactions that satisfy the given predicate will be signed. Second, our assumptions on ledger rules ensure that signed transactions involving spent UtxOs will have no effect (as opposed to something undesirable to the user).

Definition 5.3 (No Freeloading). A transaction building protocol allows **no freeloading** if clients cannot gain enough information to build a transaction without completing the protocol with a service provider.

We model the no freeloading property in two ways. First, we define the (stronger) indistinguishable until posted notion, which prevents a client from distinguishing even a single bit of an unknown UtxO. In section 7.1, in order to efficiently support multiple SPOs, we weaken this definition to allow for partial leakage but not enough to allow a client to spend an UtxO.

Definition 5.4 (Indistinguishable until posted). A transaction building protocol has the **indistinguishable until posted** (IuP) property if a PPT adversarial client $C_{\mathcal{A}}$ cannot win the following experiment with probability significantly higher than $\frac{1}{2}$.

For an honest service provider SP and any client $C_{\mathcal{A}}$ we have that: client provides 2 states $LState_0, LState_1$ and one intent $int_{post} \in I_{post}$ such that for $i = 0, 1$ it is $mkToSpec(LState_i, int_{post}) \neq \perp$. The experiment flips a coin d and runs the protocol between SP and $C_{\mathcal{A}}$ using state $LState_d$. $C_{\mathcal{A}}$ outputs d^* . The adversarial client wins iff $d = d^*$.

The indistinguishable until posted property of our protocol is derived from Theorem 5.7 described later in this section and the protocol structure. Given an adversary against $\mathcal{A}$ IuP, we can build an adversary against the weak blindness of the $WBPS[P]$ scheme by running the SP once on $LState_0$ and once on $LState_1$ to produce two challenge messages for the blindness security game and simulate the role of the challenge towards $\mathcal{A}$ by passing on the rest of the protocol messages (the experiment enforces random padding internally). If $\mathcal{A}$ can distinguish which ledger was used in producing the messages, we can use the same index as our guess in the IuP game.

5.2 (Weakly) Blind Signatures for Abstract Transactions

In order to allow the light client to sign an abstract transaction we use blind signatures on (partially) blinded transaction objects. At a minimum the SP hides the inputs to the transaction, i.e., the references to UtxO objects which are present in the ledger and required to cover the transaction. This ensures that the LC cannot simply obtain the transaction from the SP, then edit the transaction to remove the SPs compensation.

In this section, we formally introduce the notion of *weakly* blind predicate schemes (wPBS) which, similar to full PBS schemes, hide the signed message from the signer but may allow the signer to correlate signatures and signing sessions

ex poste. We also describe our weakly blind predicate scheme, a more efficient modification of the (full) PBS mechanism in [27].

Given our application, the unlinkability property of blind signatures is not of much benefit: signed transactions are somewhat infrequent, often have different payees or payment sums (i.e different predicates) and are indirectly timestamped due to their time of posting on the blockchain. Further, there seems to be little benefit in preventing the light client from knowing which protocol section produced which payment. For this reason, we do away with the requirement of the signer being unable to distinguish between messages signed on different sessions. We only require that the no freeloading property is met, which translates to the signer being unable to distinguish the message *before* the signature-message pair is posted, or—in case of the multi service provider setting—leakage is limited (see Section 7).

Our construction is inspired by the predicate blind signature (PBS) mechanism in [27] which is compatible with standard Schnorr signatures (or even EdDSA signatures² when instantiated accordingly). This is a core property as it allows our protocol to be compatible with popular blockchains such as Bitcoin and Cardano without any modifications.

Our weakening of the PBS notion (WPBPS) brings about the benefit that we can remove certain blinding operations from the protocol (for a small efficiency increase) and also from the proof of correctness of the challenge generation (where the gain is more significant as it removes group exponentiations using foreign field arithmetic). The resulting scheme is similar in operation to the Signatures over Blocks of Committed Messages construction of Bobolz et. al. [8] with the addition of predicate checking.

We adapt the definitions of [27] to account for the weak variant of blindness.

A WBPS scheme is parameterized by a family of polynomial-time-computable predicates, which are implemented by a p.t. algorithm P , the predicate compiler: on input a predicate description $prd \in \{0, 1\}^*$ and a message $m \in \{0, 1\}^*$, P returns 1 or 0 indicating whether m satisfies prd .

A WBPS scheme $WBPS[P]$ for predicate P is defined by the following algorithms. We focus on schemes with 2-round (i.e., 4-message) signing protocols for concreteness.

- $Setup(1^\lambda) \rightarrow par$: the setup algorithm, on input the security parameter, outputs public parameters par , which define a message space $\mathcal{M}_{par}$.
- $KeyGen(par) \rightarrow (sk, vk)$: the key generation algorithm, on input the parameters par , outputs a signing/verification key pair (sk, vk) , which implicitly contain par , i.e., $vk = (par, vk')$.
- $\langle Sign(sk, prd), User(vk, prd, m) \rangle \rightarrow (b, \sigma)$: an interactive protocol with shared input par (implicit in sk and vk) and a predicate prd is run between the signer and user. The signer takes a secret key sk as private input, the user's private input is a verification key vk and a message m . The signer outputs $d = 1$ if the interaction succeeds and $d = 0$ otherwise, while the user outputs a signature σ if it succeeds, and $\perp$ otherwise.
- $Ver(vk, m, \sigma) \rightarrow 0/1$: the (deterministic) verification algorithm, on input a verification key vk , a message m and a signature σ , outputs 1 if σ is valid on m under vk and 0 otherwise.

²More concretely, signature derivation is non-standard, but verification is unchanged.

For a 2-round protocol the $\langle \text{Sign}(sk, prd), \text{User}(vk, prd, m) \rangle \rightarrow (d, \sigma)$ interaction can be realized by the following algorithms:

$$\begin{aligned}
 (txt_{U,0}, st_{U,0}) &\leftarrow \text{User}_0(vk, prd, m) \\
 (txt_{S,1}, st_S) &\leftarrow \text{Sign}_1(sk, prd, txt_{U,0}) \\
 (txt_{U,1}, st_{U,1}) &\leftarrow \text{User}_1(st_{U,0}, txt_{S,1}) \\
 (txt_{S,2}, d) &\leftarrow \text{Sign}_2(st_S, txt_{U,1}) \\
 \sigma &\leftarrow \text{User}_2(st_{U,1}, txt_{S,2})
 \end{aligned}$$

We use $txt_{X,i}$ to represent the output of participant X in round i and $st_{X,i}$ to represent the internal state of participant X in round i , omitting i when it is not needed. As shorthand, we write $(d, \sigma) \leftarrow \langle \text{Sign}(sk, prd), \text{User}(vk, prd, m) \rangle$ for the above sequence.

Definition 5.5. A WBPS scheme $\text{WBPS}[P]$ satisfies weak blindness if for all p.p.t. adversaries $\mathcal{A}$:

$$\begin{aligned}
 Adv_{\text{WBPS}[P], \mathcal{A}}^{\text{BLD}}(\lambda) &:= \Pr[\text{BLD}_{\text{WBPS}[P]}^{\mathcal{A},1}(\lambda) - \text{BLD}_{\text{WBPS}[P]}^{\mathcal{A},0}(\lambda)] \\
 &\text{is negligible in } \lambda.
 \end{aligned}$$

We note that the BLD experiment mauls the message by appending a random bitstring r of length λ . This is appropriate to our setting, where the SP is explicitly allowed to use the “notes” field of a transaction to add a randomizer. Without this mauling, any indistinguishability-based definition would fail.

$\text{BLD}_{\text{WBPS}[P]}^{\mathcal{A},b}(\lambda)$ $par \leftarrow \text{Setup}(1^\lambda)$ $(m_0, m_1, prd, vk', st_{\mathcal{A}}) \leftarrow \mathcal{A}_1(par)$ if $P(prd, m_0) = 0$ or $P(prd, m_1) = 0$ then return 0 $aux_{nt} \leftarrow \{0, 1\}^\lambda$ $m \leftarrow m_b aux_{nt}$ $vk \leftarrow (par, vk')$ $sess \leftarrow \text{init}$ $b^* \leftarrow \mathcal{A}_2^{\text{ChalUser}}(st_{\mathcal{A}})$ return $b = b^*$	$\text{ChalUser}(msg = \emptyset)$ if $sess = \text{await}$ $sess \leftarrow \text{closed}$ $\sigma \leftarrow \text{User}_2(st_u, msg)$ if $sess = \text{closed}$ $msg' \leftarrow \text{Ver}(vk, m, \sigma)$ if $sess = \text{open}$ $sess \leftarrow \text{await}$ $(msg', st_u) \leftarrow \text{User}_1(st_u, msg)$ if $sess = \text{init}$ $sess \leftarrow \text{open}$ $(msg', st_u) \leftarrow \text{User}_0(vk, prd, m)$ return msg'
---	---

Fig. 13. Weak Blindness experiment $\text{BLD}_{\text{WBPS}[P]}^{\mathcal{A},b}(\lambda)$ for a WBPS scheme with predicate compiler P , adversary $\mathcal{A}$ and parameter b .

Definition 5.6. A WBPS scheme $\text{WBPS}[P]$ satisfies unforgeability if for all p.p.t. adversaries $\mathcal{A}$

$$Adv_{\text{WBPS}[P], \mathcal{A}}^{\text{EUF-CMA}}(1^\lambda) := \Pr[\text{CMA}_{\text{WBPS}[P]}^{\mathcal{A}}(1^\lambda) = 1] \text{ is negligible in } \lambda.$$

5.3 WBPS Scheme

We present the full weakly blind predicate Schnorr signature scheme (wPBS) in Figure 15.

$\text{CMA}_{\text{WBPS}[P]}^{\mathcal{A}}(\lambda)$ <pre> $par \leftarrow \text{Setup}(1^\lambda); \quad (sk, vk) \leftarrow \text{KeyGen}(par)$ $Q \leftarrow 0; \quad S \leftarrow 0; \quad P \leftarrow 0$ $(\vec{m}^*, \vec{\sigma}^*, \vec{prd}^*) \leftarrow \mathcal{A}^{\text{SigInit, SigComplete}}(vk)$ $n \leftarrow \vec{m}^*$ if $\prod_i \text{Ver}(vk, m_i^*, \sigma_i^*) \neq 1$ return 0 if $\prod_i \text{prd}_i^*(m_i^*) \neq 1$ return 0 if $\exists i, j : (i \neq j) \wedge (m_i^*, \sigma_i^*) = (m_j^*, \sigma_j^*)$ return 0 if $n > Q$ return 1 if $\nexists \rho \in \text{Perm}(S) :$ $\forall i \leq n : (\text{prd}_{\rho(i)}(m_i) = 1) \wedge (st_{\rho(i)} = \perp)$ then return 1 return 0 $\text{SigInit}(\vec{prd}, msg_{U,0})$ $S \leftarrow S + 1$ $P \leftarrow P + 1$ $(msg, st_S) \leftarrow \text{Sign}_1(sk, \vec{prd}, msg_{U,0})$ $\vec{prd}_S \leftarrow \vec{prd}$ return msg $\text{SigComplete}(s, msg_{U,1})$ if $s > S$ or $st_S = \perp$ then return $\perp$ $(msg, d) \leftarrow \text{Sign}_2(st_S, msg_{U,1})$ if $d = 1$ then $Q \leftarrow Q + 1$ $P \leftarrow P - 1$ $st_S \leftarrow \perp$ return msg </pre>	
--	--

Fig. 14. Chosen Message Unforgeability Experiment $\text{CMA}_{\text{WBPS}[P]}^{\mathcal{A}}(\lambda)$ for a WBPS scheme with predicate compiler P , adversary $\mathcal{A}$

5.4 Security

THEOREM 5.7. *The WBPS[P] scheme of Figure 15 achieves weak blindness when the encryption scheme PKE is IND-CPA secure, NArg has the zero knowledge property, assumption 2 holds w.r.t. the hash generator HGen.*

PROOF. We structure our proof as a sequence of games so that (1) two consecutive games are computationally indistinguishable to the adversary and (2) in the final game, the view of the adversary is independent of the challenge b . We define the initial game G_0 to be $G_0 := \text{BLD}_{\text{WBPS}[P]}^{\mathcal{A},b}(\lambda)$. For G_1 we replace proofs with simulated ones. This is not detectable by the adversary due to the zero knowledge property of NArg. Second, for game G_2 we replace C with an encryption of a fixed message $\bar{m}$ rather than m_b , this too is undetectable³ due to the IND-CPA security of PKE. For game G_3 we replace the challenge c to also use $\bar{m}$ rather than m_b in the hash. This is computationally indistinguishable due to assumption 2. $\square$

THEOREM 5.8. *The WBPS[P] scheme of Figure 15 achieves unforgeability when the encryption scheme PKE is IND-CPA secure, NArg is sound, and assumption 1 holds w.r.t. the hash generator HGen.*

PROOF. We follow the proof of [27] with little changes. As before, we structure our proof as a sequence of games so that the probability of adversarial success changes negligibly from one game to the next. We define the initial game G_0 to be $G_0 := \text{CMA}_{\text{WBPS}[P]}^{\mathcal{A}}(\lambda)$.

³This will also change the input of the simulator, but this is not an obstacle: we simply consider the simulator to be part of the adversary we are constructing against the IND-CPA security of PKE.

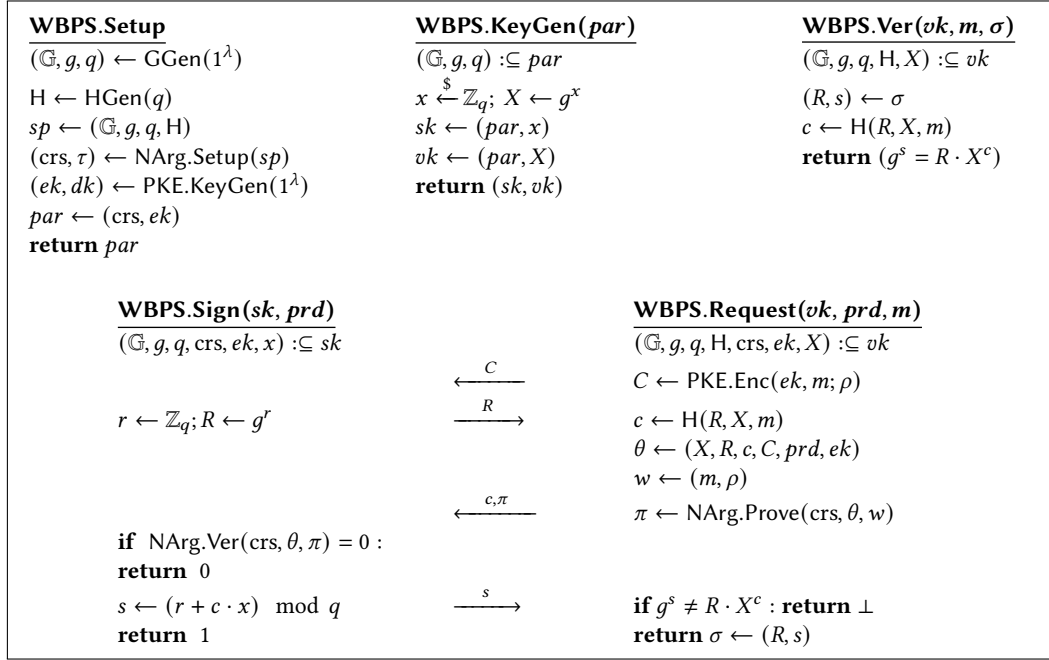


Fig. 15. The weakly blind predicate Schnorr signature scheme WBPS[P]

For game G_1 , the experiment holds the decryption key for PKE, so that we extract the message from C ahead of time, and thus predict the c value once the value of R has been determined. If the value sent by the adversary is different from the predicted one, and the accompanying proof π verifies we abort early, diverging from the original game. This only happens with negligible probability though, due to the adaptive soundness of NArg (if π does not verify, G_0 would abort as well).

For game G_2 , we check to see if any of the adversarial signatures uses a message that belongs to a session that was not closed. If so, we abort early, diverging from G_1 . However, this implies that the second part of the signature s along with the secret key x can be used to calculate the discrete log of R . Thus, if the early abort occurs (i.e. adversary manages to win using unfinished sessions) with non negligible probability we can build a discrete log calculator by embedding a discrete log challenge in one of the R values. Otherwise, we can assume that almost all of the adversarial wins use at least one “new” message, and thus the success probability for G_1 and G_2 differ only negligibly.

For G_3 we build a reduction to the EUF-CMA security of Schnorr (Assumption 1). We remove key generation from the challenger, and instead obtain a vk from the Schnorr EUF-CMA security game. When we decrypt a message m , we query it on our signing oracle, obtaining R, s as the signature. The reduction forwards R to the adversary anticipating c, π . The checks introduced in G_1 ensure the c value matches the predicted one, and thus s correctly completes the signature. This ensures the simulation can complete the game. If the adversary wins the WBPS[P] security game, there must be a signature on a new message that the reduction uses to also win the Schnorr EUF-CMA game with the same probability. $\square$

6 Cavefish: Communication-optimal Transaction Building Protocol

We give an overview of our light client protocol and its components. As the protocol is uniquely suited for UTxO-based blockchains, we develop our approach to support the general ledger model explained in Section 3.6 and the intents set forth in Section 4.

6.1 Security Model and Requirements

Rational Actors. For liveness with a single service provider, we assume that protocol participants are rational in the sense that they will opt to complete tasks that benefit them directly (i.e. in the form of rewards, or in the form of a desired transaction being posted). For safety, we assume parties can be fully malicious. We will relax the rational requirement for liveness in Section 7 by introducing multiple SPs.

Transaction Expiry. For safety, we require that transactions can be set to be valid only within a certain time window. This is to prevent trivial attacks where a malicious SP delays posting a transaction indefinitely. The user could contact a different SP to create a second transaction with similar parameters (e.g. by paying the same recipient with different coins) with the risk that the first SP would also post their transaction, making the user pay twice.

Idempotent Double-spends. For safety, we also require that attempted double-spends are idempotent, i.e., they are simply not valid transactions and do not penalize (slash) the user for signing them.

Explicit Balancing. We require that the transaction specification allows explicit calculation of the value balance of inputs - outputs, commonly referred to as a “tip” to miners or validators. E.g., in Cardano, the tip value is part of the transaction data, and in Bitcoin (post SegWit) it can be calculated from the outputs and the signing data. In pre-SegWit Bitcoin, the calculation was implicit: transaction and signature data only included output values.

Trusted Setup. We specify a proof system using a trapdoor, which is a common characteristic of efficient options such as Groth16 [31] or Plonk [29]. It is possible to use alternatives featuring transparent setup with some impact to communication size. In the case of a transparent setup for NArg, we still need to generate keys for PKE. In the case of ElGamal in the random oracle model, this can be accomplished by hashing an unpredictable string into a group element.

6.2 Protocol Description

The protocol operates as follows: the light client connects to the SP and sends a posting intent int_{post} , as well as a public key that has received payments (and, optionally a BIP-032 chaincode so that the SP can also derive child keys from the initial one—we explain the details in Section 6.4). The SP will then search for UTxOs belonging to the client and form a transaction that satisfies the intent int_{post} , including in the transaction a tip for the SP’s own work. Lastly, the SP adds a random note aux_{nt} to the note field of the transaction. Then, the client and SP engage in a weakly blind predicate signature protocol so as to efficiently have the client sign the transaction whilst ensuring that (1) the client needs to sign before learning the full transaction (typically the used UTxOs are hidden), and (2) that the signed transaction satisfies int_{post} by means of satisfying the corresponding predicate.

Our scheme operates as follows: given an intent (i.e. predicate description) $prd := int_{post}$ and predicate compiler P such that $(P(prd))(tx) := \text{chkSpec}(int_{post}, \text{mkAbs}(tx))$ the requestor (i.e. the SP) encrypts the message, in our case the full transaction $m := tx$, and sends the commitment C to the signer, who returns a random group element R used

in the final Schnorr signature. The requestor replies with the Schnorr challenge c as well as a proof π of its correct construction, and of the fact that tx satisfies int_{post} .

We instantiate the scheme with a parametrisable NArg for the following relation,

$$\text{R}_{\text{Cavefish}}(\underbrace{(q, \mathbb{G}, G, H)}_{\text{parameters } par}, \underbrace{(X, R, \text{com}_{\text{tx}}, \text{TxAbs}, c, \text{int}_{\text{post}})}_{\text{known statement } \theta}, \underbrace{(\text{tx} || \text{aux}_{\text{nt}}, \rho)}_{\text{witness } \omega}) \quad (1)$$

checking that $\text{chkSpec}(\text{int}_{\text{post}}, \text{TxAbs}) = 1 \wedge m = \text{tx} || \text{aux}_{\text{nt}} \wedge C = \text{PKE}.\text{Enc}(m; \rho) \wedge c = H(R, X, m) \wedge \text{TxAbs} = \text{mkAbs}(\text{tx})$. The signer (LC) completes the signing only if the proof verifies correctly.

Light and Extended Versions. We showcase our protocol for two kinds of intents: “light” and “extended”. In the light version, we only support intents where the transaction abstract does not give the light client any information the SP does not wish to disclose. In turn, this makes verification simpler. We can disclose the transaction abstract to the client in the open, and only need to check that the abstract was “filled” correctly in zero knowledge. By design, the abstract hides the input transactions, but it could reveal, e.g. some state details of a smart contract.

A practical example of “light” intents is

$\text{int}_{\text{light}}(\text{tx}) := \{\text{return}(\text{makeAbs}(\text{tx}) = \text{makeAbs}(\text{tx}_0))\}$, i.e. the light client fully determines the requested transaction (including SP fee) to be tx_0 , apart from the input UTXOs (which the LC does not know) and the note field aux which is free to the SP to set. This implies that TxAbs provides no new information to the light client. This reduces the computational effort by SP invested in running the NArg prover as chkSpec consists only of substring equality checks. We note here that no range check is necessary for values: makeAbs (Section 4) retains the prescribed value assigned to all outputs except the first (which is used as a “change” address), i.e., the only option for the SP is to assign all excess value to the change address.

For the extended version, we alter the predicate compiler to $(P_{\text{ext}}$ such that $(P_{\text{ext}}(\text{prd}))(\text{tx}) := \text{chkSpec}(\text{int}_{\text{post}}, \text{tx})$, i.e., we check tx instead of $\text{mkAbs}(\text{tx})$. Consequently, we set $\text{makeAbs}(x) := \{\text{return } 1\}$, and change the predicate check from $\text{chkSpec}(\mathcal{I}_{\text{post}} \times \text{TxAbs}) \rightarrow \{0, 1\}$ to $\text{chkSpec}(\mathcal{I}_{\text{post}} \times \text{Tx}) \rightarrow \{0, 1\}$. This ensures no information is leaked by TxAbs . Our results hold for both variants.

We show our complete light client protocol in Figure 16, leveraging the weakly blind signatures predicate we described in Section 5.3. For simplicity, the figure describes the process for a single UTXO signature; if more are needed, multiple WBPS sessions can be run sharing the same com_{tx} .

6.3 Security

THEOREM 6.1. *If the ledger rules include the idempotent doublespends property and $\text{WBPS}[P]$ is unforgeable, the protocol of Section 6.2 is safe.*

PROOF. The safety property is derived from two factors: first, the unforgeability property of $\text{WBPS}[P]$ (Theorem 5.8, Section 5.4) will ensure that only transactions that satisfy the given predicate will be signed. Second, our assumptions on ledger rules ensure that signed transactions involving spent UTXOs will have no effect (as opposed to something undesirable to the user). $\square$

THEOREM 6.2. *If $\text{WBPS}[P]$ scheme is weakly blind, the protocol of Section 6.2 is indistinguishable until posted.*

PROOF. The indistinguishable until posted property of our protocol is derived from Theorem 5.7 (Section 5.4) and the protocol structure. Given an adversary against $\mathcal{A}$ PuP, we can build an adversary against the weak blindness of

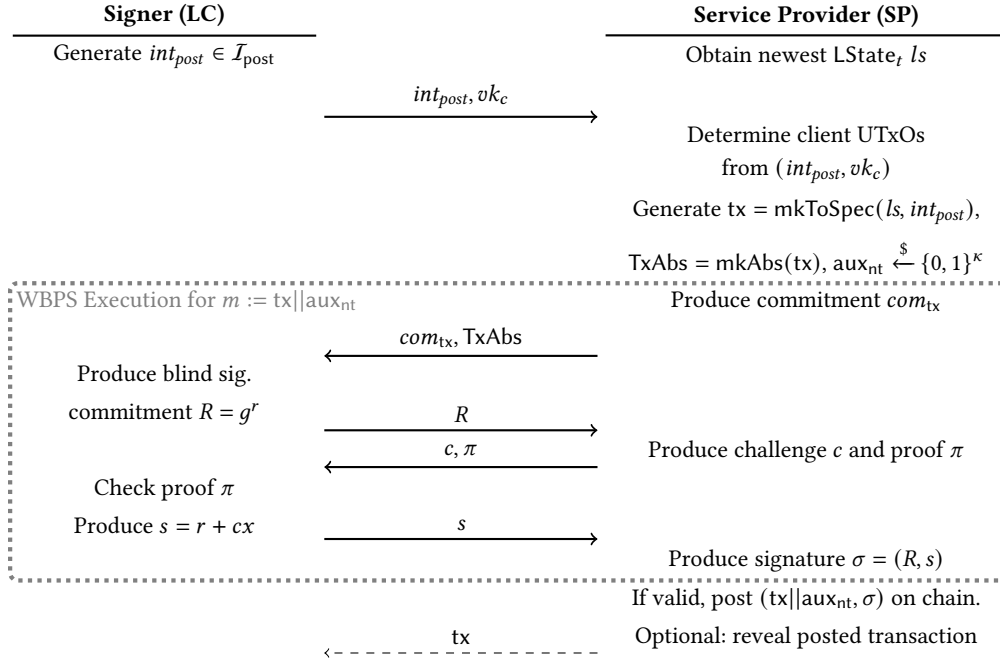


Fig. 16. Cavefish protocol featuring WBPS scheme. A detailed description of the WBPS scheme is in Fig. 15.

the WBPS[P] scheme by running the SP once on $LState_0$ and once on $LState_1$ to produce two challenge messages for the blindness security game and simulate the role of the challenge towards $\mathcal{A}$ by passing on the rest of the protocol messages (the experiment enforces random padding internally). If A can distinguish which ledger was used for the messages we use the same index as our guess in the Pul game. $\square$

6.4 Analysis & Extensions

Storage and Communications Requirements. We point out that our protocol is optimal in terms of light client storage and light client & SP communication. The light client (LC) only needs to store information about which wallet subkeys have been active. In terms of communication, our protocol transmits only a constant number of elements when using a succinct proof system. This compares favorably to solutions such as SPV which incur linear costs or even solutions based on succinct proof systems for the chain tip, but where the LC needs to verify transaction inclusion into blocks via Merkle proofs.

Transaction Discovery via HD Wallets. Assuming that the client only uses a single address can be unrealistic, as requiring the LC to send the SP a list of addresses presumed active. Our protocol is compatible with BIP-32 [57] hierarchical deterministic (HD) wallets. This enables child addresses to be deterministically created from a parent address by hashing the parent key X , a chaincode value cc and an address index i . Concretely, $o_i := H(X, cc, i)$ is the private key offset of child key i , i.e. the i -th child keypair is $x_i = x + o_i$ and $X_i = X \cdot g^{o_i}$. Child chaincodes are determined in the same way.

Thus, an LC can simply transmit the chaincode corresponding to its public key X , and the SP will be able to derive a list of child addresses (bounding indexes can be done heuristically, or with hints from the client). The protocol then proceeds with minor changes:

- (1) The challenge c is now derived as $c = H(R, X_c, m)$
- (2) The witness w now includes the index of the child key i (or vector of indices i and chaincodes $\mathbf{cc}$ if the child is further down the tree).
- (3) The relation now checks that the hashed value is of the form $X_c = X \cdot g^{o_c}$ where $o_c = \text{HDDerive}(X, i, \mathbf{cc})$.
- (4) The SP modifies the component $s := xc + r$ to $s' = s + o_c \cdot c$, adjusting for the child offset.

Notably, the LC does not learn which child address was used, it can simply sign with respect to the sent public key X and the SP can maul the signature as needed.

Optional Final Protocol Step. The SP is incentivized to submit the LC's transaction to the network by the fee they earn. However, the submission may not be visible to the LC. We may add the following additional step to our protocol: Once the SP observes that the LC's transaction has been included in the chain state (with some level of certainty), the SP sends the full transaction to the LC as evidence that the protocol has been completed. The SP does not lose anything by sharing a transaction that is included in the chain because in the UTxO model, an already-included transaction is guaranteed to be rejected when resubmitted. Even if the LC were to modify the transaction at this stage (e.g., to remove the SP's fee), submitting a new transaction containing inputs of an already-included transaction would still cause the new transaction to be rejected.

7 Multi-SP Model and Transaction Optimality

We present a Multi-SP variant of Cavefish that mitigates some of the limitations that arise when relying on a single SP; most notably, the resulting transactions might be suboptimal in the economic sense, and a single SP inevitably provides lower liveness guarantees. Revealing the posted transaction (as described above) helps, but only a Multi-SP approach can provide liveness when facing malicious SPs that purposely go silent during/after protocol execution.

Suboptimal Transactions. In response to an LC query, it is possible for an SP to construct a *sub-optimal* transaction (in terms of cost) which nevertheless matches the LC's specification. For example, the LC may include a larger-than-necessary system fee in the transaction or not provide the LC with the best available exchange price for a specific token. We consider cost optimizations an orthogonal issue that can be addressed by e.g., a market system. Another option is for the SP to cover the transaction fee for the LC, including it in their advertised service price.

Liveness against Malicious SPs. In the single SP model, correctness and liveness are essentially the same notion: there is only a single SP so if it is honest the protocol is live. Otherwise, liveness is impossible with no honest SPs (the adversary can simply be silent). However, in a more realistic setting, the client will end up running the protocol with a single SP chosen from a larger pool that potentially contains malicious as well as honest ones.

Our goal in the multiple SP setting is not only to achieve liveness, but to also avoid suboptimal transactions, given enough honest SPs. This is not trivial: simply selecting SPs at random (protected by the safety property) until we get an honest one will not work.

Malicious SPs are able to force long delays for the user by completing the signing but never posting the transaction. This forces the user to wait until the signed transaction is no longer valid before retrying (see Section 6). Otherwise, as

intents are not necessarily uniquely satisfied, it is possible that the first (malicious) node may be able to post the first transaction (which was purposefully delayed) after the posting of the second (honest) one.

This can cause the LC to, e.g. double pay for a service as well as the SP's fee. Even so, the client can blacklist any SP that causes a delay, and retry after the validity period has passed. Given enough retries, the LC will reach an honest SP and the transaction will be posted.

7.1 Multi-SP Protocol Variant

We can improve both liveness and optimality by making a small adjustment that turns Cavefish into a multi-SP aware protocol. The core idea for this design is to ensure that honest SPs will commit to the same candidate transaction. If a malicious SP tries to follow the honest SPs but force a delay, the client will be able to resume with any honest SP without risking double payment. If a malicious SP does not follow the honest SPs in the candidate transaction, they will be outvoted. To ensure that honest SPs will agree with each other, we assume transaction construction is deterministic and only makes use of the settled part of the ledger.

Definition 7.1 (Liveness under honest majority). A transaction protocol is **live** if for an honest light client, a set of service providers $\mathcal{SP}$ such that the majority of $\mathcal{SP}$ s is honest, and any state $LState_i \in LState$ and intent $int_{post} \in \mathcal{I}_{post}$ so that: if $mkToSpec(LState_i, int_{post}) \neq \perp$, then the protocol completes and transaction Tx' produced by the SP is s.t. $checkTxL(LState_i, Tx') = 1$.

We also need to slightly weaken our indistinguishable until posted (IuP) property to private until posted (PuP). That is, a client will not be able to produce a full transaction without completing the protocol. In order to model the client's lack of knowledge, we have the experiment construct transactions which "pay" the adversary, and require the adversary to successfully spend a UTxO from one of these transactions before the protocol completes in order to win.

Definition 7.2 (Private until posted). A transaction building protocol has the **private until posted** (PuP) property if a PPT adversarial client $C_{\mathcal{A}}$ cannot win the following experiment with non-negligible probability.

For an honest service provider $\mathcal{SP}$ and any client $C_{\mathcal{A}}$ we have that: client provides an initial ledger state $LState$, n pairs of public keys $PubKey_0, \dots, PubKey_n$, and values $v_1, \dots, v_n$ as well as a source keypair $PubKey_s, sk_c$. If the key pair $PubKey_s, sk_c$ is invalid, or if $LState$ does not have UTxOs addressed to $pubkey_s$ of value at least $\sum v_i$, the experiment concludes and the adversary loses. The experiment then produces a followup ledger $LState'$ by adding transactions with $LState$ accordingly so that new UTxOs are created, addressed to the specified public keys and with the corresponding values.

$C_{\mathcal{A}}$ posts an intent $int_{post} \in \mathcal{I}_{post}$. If $mkToSpec(LState', int_{post}) = \perp$, the adversary loses and the experiment stops. The experiment runs the protocol between $\mathcal{SP}$ and $C_{\mathcal{A}}$ using state $LState$. $C_{\mathcal{A}}$ outputs Tx^* . If the protocol completes, the adversary loses and the experiment stops. At any point before that, the adversary may halt the protocol and instead output a transaction Tx^* . The adversarial client wins iff Tx^* is valid for $LState'$ and spends at least one UTxO not in the initial $LState$.

Ensuring only one signed transaction is posted. In the multi-SP variant of the protocol, the LC might end up signing multiple transactions if the first attempt stalls. In fact, the LC may sign any transaction meeting the requirements described in the intent. The multi-SP protocol guarantees that only one signed transaction can eventually be posted on the ledger like the single SP protocol. Since several signed transactions possibly reach the consensus layer of the underlying blockchain platform, we need to guarantee that at most one of them will be admitted to the ledger. We

accomplish this by ensuring via a fingerprint that the input sets of all signed transactions intersect. Recall here that in UTxO-based ledgers, a transaction is not valid unless all its inputs are present in the ledger state to which it is applied, and an invalid transaction is never partially applied. Therefore, if one transaction from a set of valid transactions with intersecting inputs has been applied to a given state, no other transactions in the set will be able to make any state changes to the resulting updated state (this includes paying fees, etc.).

We now formulate in brief a version of our protocol where the LC instead sends the specification to multiple SPs, selecting the best response, and engaging in the rest of the protocol only with that SP.

7.2 Augmenting the NArg for Multi-SP

During setup, we select a family of pseudo random permutations F , and then, during the second step, after receiving the intent from the client, and creating the transaction Tx each service provider will calculate a fingerprint $f \leftarrow F_k(\text{Tx})$ of the transaction, where the key k is the last input of Tx i.e. $k := \text{Tx.inputs}[-1]$. Note that Tx does not yet include the aux_{nt} field.

The client receives all the fingerprints and discards all except the most popular one. As we assume an honest majority and deterministic transaction construction, this fingerprint will be the one produced by all honest SPs. Then, it chooses at random one of the SPs that provided the most popular fingerprint value, and initiates the WBPS protocol with it.

The protocol proceeds with a small modification to the NArg relation (see Equation 1):

$$R_M(\text{par}, (f, \theta), (\text{tx}, \text{aux}_{\text{nt}}, \rho)) := (F(\text{tx}) = f) \wedge R_{\text{Cavefish}}(\text{par}, \theta, (\text{tx} || \text{aux}_{\text{nt}}, \rho))$$

THEOREM 7.3. *If mkToSpec is deterministic and F is sampled from a family of secure pseudo-random functions, the protocol of Section 7.2, instantiated with the WBPS[P] scheme of Section 5.3 is private until posted.*

PROOF. (sketch) We will prove this in a series of games. The experiment will first create a fresh keypair PubKey_x, sk_x and create transactions paying into PubKey_x from the adversarial source address PubKey_s, sk_s provided to the experiment. It then posts individual transactions paying into the pubkeys requested by the adversary. The protocol then proceeds as normal. This defines the initial game, G_0 . For the first hybrid, G_1 , we invoke the zero-knowledge of the proof system to swap out the proof given to the adversary with a simulated one. Then, for G_2 , we invoke the hiding property of the commitment scheme to swap out com_{tx} for a commitment to 0. Finally, for the game G_3 we invoke the pseudorandomness of F to swap out the fingerprint with a random string. By construction, G_3 provides no information to the adversary about any of the new transactions, and the change in adversarial behaviour between G_3 and G_0 is negligible. $\square$

THEOREM 7.4. *If the ledger rules include the Idempotent Doublespends property and WBPS[P] is unforgeable, the protocol of Section 7.2 is safe. Additionally, if multiple valid transactions are produced by different SPs, they must be mutually exclusive. That is, if there are two output transactions Tx_0, Tx_1 where $\text{checkTxL}(\text{LState}, \text{Tx}_i) = 1$ and $\text{chkSpec}(\text{int}_{\text{post}}, \text{Tx}_i) = 1$ then it must be that $\text{checkTxL}(\text{LState}_{1-i}, \text{Tx}_i) = 0$ where LState is the initial ledger state, and LState_i is LState after applying Tx_i .*

PROOF. (sketch) The safety proof is identical to Theorem 6.1. For the additional property, if there exist two or more output transactions then they must all share the same fingerprint f for the LC to proceed.

The fingerprint f is perfectly binding since F_k is a permutation. Thus, either all transactions sharing f use the same UTxO set, or we derive a WBPS[P] forgery, which happens with negligible probability. If the transactions use the same

input UTxO set, the semantics of `updateUTxO` (Section 3.6) imply that the first transaction to be applied to `LState` will consume its input set, invalidating all others. $\square$

8 Implementation

To assess the efficiency of our construction, we implement and benchmark the NArg component of the protocol. Given today’s implementations of zk-SNARKs, the NArg is expected to be the most time and resource intensive part of Cavefish. We base our tests on the trusted-setup zk-SNARK system *Groth16* [31] implemented by *Iden3* [34]. The circuits we construct and benchmark are inspired by the implementation of [27] and written in the domain-specific language of Circom 2.1. The NArg we implement captures the relation R_{Cavefish} (see Section 6.2). We test the “light” version of Cavefish, i.e., the intent is given by $int_{\text{light}}(tx) := \{\text{return } (mkAbs(tx) = mkAbs(tx_0))\}$ and effectively specifies the transaction, except for the input UTxOs and note field aux_{nt} .

8.1 Implementation Choices

To measure the arithmetic complexity of the relation R_{Cavefish} , we use BN254 as the curve for *Groth16* to operate on, i.e., the curve group has 254 bits and the relation is instantiated over an arithmetic circuit with modulus of 254 bits. The BN254 curve is one of the standard choices in Circom and forces the inputs to the circuit to be elements of the field given by BN254. We capture transaction tx and abstract transaction tx_A as bit strings of some length n , encoded as field elements, which allows us to instantiate a circuit that can handle transactions of length up to n .

To implement the encryption scheme PKE that is needed to encrypt tx , we use ElGamal [25] public key encryption over the Baby-JubJub curve [56]. The reason for choosing Baby-JubJub curve is that the field given by BN254 is the base field of the Baby-JubJub curve and any element can be represented as two elements of the BN254 field. Therefore, the group operation is efficiently arithmetizable in the circuit [27]. In addition, we encapsulate key and encryption of ElGamal using DHIES [1], i.e., the shared secret in ElGamal serves as a seed to the PRF generating random group elements for an additive one-time-pad that encrypts tx . The PRF is instantiated as a Poseidon hash [30] that is efficiently arithmetizable by design.

To be compatible with existing blockchain platforms and create standard Schnorr signatures, we have to adopt the hash function(s) specified by the respective ledger, despite most of them not being “circuit-friendly”. We use the library of hash functions in [38] which provides Circom implementations for many popular hash functions. The authors put effort into the optimization, but more efficient implementations could be possible. Nevertheless, the complexity of our resulting circuits is mainly governed by the number of rounds the hash function has to execute when producing the Schnorr challenge.

8.2 Benchmarks

We test partially blind signatures for Bitcoin and Cardano, i.e., the “light” version in Section 6.2. This version of Cavefish uses int_{light} which can be implemented with substring equality checks, provided the transaction to be signed can be unambiguously split into blinded and nonblinded parts. Cardano transactions are CBOR-encoded and contain the transaction-id and index for every UTxO serving as input [35]. Similarly, Bitcoin transactions (after Taproot [23]) feature an input count followed by a list of inputs in binary format, each consisting of transaction hash and output index. We assume for our benchmarks that one UTxO acts as input to the transaction and needs to be blinded.

We capture the number of constraints, the proving key size, and the proof size that R_{Cavefish} requires in Circom. The number of constraints are obtained from the arithmetization when given as R1CS relation. We also measure the time it

Table 1. Benchmark of R_{Cavefish} implementing the “light” version of Cavefish.

	Bitcoin	Cardano
Signature scheme	Schnorr	EdDSA
Curve + Hash function	Secp256k1 + SHA-256	Ed25519 + SHA-512
Transaction size	254B (288b blinded)	285B (333b blinded)
Proving key size	115 MB	116 MB
Proving key verif. time	18.6 s	20.5 s
Verification key size	67 kB	93 kB
Proving time / Proof verif. time	5.1 s / 0.57 s	5.8 s / 0.59 s
Proof size	806 B	805 B
Number of constraints	226509	245181

takes to create the resulting circuit, the proving time and the proof verification time. Our results are summarized in Table 1 for Bitcoin and Cardano as the target platform. The experiments were executed on commodity hardware based on an Intel(R) Core(TM) i7-8750H CPU at 2.20 GHz with 12 cores and 16 GB of RAM.

Bitcoin has a conceptual difference from Cardano as the message being signed by the Schnorr signature scheme is the hash of the transaction $H(tx)$ instead of the transaction itself, i.e., $WBPS[P]$ is executed with $m := H(tx || aux_{nt})$. A direct adaption to $WBPS[P]$ that supports hashed transactions is to allow the predicate P to accept an additional input which is a witness attesting to the signed message m satisfying P . This additional witness must be included in the witness ω for R_{Cavefish} [27]. Compared to Cardano, the circuit for Bitcoin has to invoke the hash function (SHA-256) once more. On the other hand, the hash function used by Cardano (SHA-512) has a larger codomain and higher complexity. We remark that the projects we build upon (e.g., the hash function library) are in development and lack certain optimizations. Nevertheless, the introduction of a weak variant of blindness allowed us to obtain results within practical bounds and shows Cavefish is deployable in the real world.

9 Discussion and Conclusion

9.1 UTxO vs. Account-based Light Clients

Traditional UTxO-based LCs need to “prove” to the user that certain transaction(s) or blocks were included in the chain to verify validity and rule out double-spends. Account-based LCs, meanwhile, aim to verify (using proofs of (global) account state, such as state trie proofs [36]) that an account state (balance, nonce, storage) is valid at some block. The nature of the UTxO model makes it impossible for an (intent-satisfying) transaction to have unexpected consequences when applied to a ledger state [35], even when constructed based on an outdated copy. This is not the case for account-based models, where the application outcome depends sensitively on the current ledger state. What makes our protocol a challenge to adapt to account-based systems is that it proves nothing about the current ledger state to the client, thus allowing the possibility of unexpected outcomes for the LC.

9.2 Comparing Cavefish to Traditional UTxO-based Light Clients

We compare Cavefish with existing light clients (LCs) to outline the trust assumptions and attack vectors. Cavefish is a two-party protocol between a user and a service provider (SP) for transaction creation with minimal information. This contrasts with most existing LCs that have the goal of synchronizing with the chain in a resource-constrained

Table 2. Attacks affecting light clients.

	Traditional syncing LC	Cavefish
Eclipse attack	Vulnerable if relying on single full node (mitigation: multiple nodes, P2P network)	Depends whether SP is affected or if no SP is reachable
Invalid chain head	LC has to sync to chain, but with limited view compared to full node	Reduces to security of underlying chain and if SP is malicious itself
Long-range attack	More vulnerable than full node due to limited view of history and smaller committees constructing proofs for LCs	Reduces to security of underlying chain

manner and then create a transaction themselves. Both designs require a trusted genesis block and reliable consensus (safety and liveness) of the underlying chain. Since Cavefish is executed between a signer and a service provider (SP), the susceptibility to attacks against the consensus mechanism directly reduces to the security of the underlying chain and whether the SP itself is affected. This is different from traditional LCs especially if they make use of simplified synchronization mechanisms or rely on smaller validator/committee sizes compared to full nodes; in those cases LCs provide fewer guarantees than full nodes. On the other hand, Cavefish requires an honest and reliable SP to function correctly. If the SP is malicious or operates with an incomplete copy of the ledger, the generated transaction will be invalid, and Cavefish again relies on the underlying consensus mechanism and chain to discard the transaction (that could be a double-spend, for instance).

References

- [1] Michel Abdalla, Mihir Bellare, and Phillip Rogaway. 1999. DHIES: An Encryption Scheme Based on the Diffie-Hellman Problem. *IACR Cryptol. ePrint Arch.* 1999 (1999), 7.
- [2] Frederik Armknecht, Ghassan Karame, Malcom Mohamed, and Christiane Weis. 2024. Practical Light Clients for Committee-Based Blockchains. arXiv:2410.03347 [cs.CR] <https://arxiv.org/abs/2410.03347>
- [3] Foteini Baldimtsi, Varun Madathil, Alessandra Scafuro, and Linfeng Zhou. 2020. Anonymous Lottery In The Proof-of-Stake Setting . In *2020 IEEE 33rd Computer Security Foundations Symposium (CSF)*. IEEE Computer Society, Los Alamitos, CA, USA, 318–333. doi:10.1109/CSF49147.2020.00030
- [4] Bellare, Namprempre, Pointcheval, and Semanko. 2003. The one-more-RSA-inversion problems and the security of Chaum’s blind signature scheme. *Journal of Cryptology* 16 (2003), 185–215.
- [5] Fabrice Benhamouda, Tancrede Lepoint, Julian Loss, Michele Orrù, and Mariana Raykova. 2020. On the (in)security of ROS. *Cryptology ePrint Archive*, Paper 2020/945. doi:10.1007/s00145-022-09436-0
- [6] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. 2012. High-speed high-security signatures. *Journal of Cryptographic Engineering* 2, 2 (01 Sep 2012), 77–89. doi:10.1007/s13389-012-0027-1
- [7] Bitcoin Project. 2025. Bitcoin — Open source P2P money. <https://bitcoin.org/>. Accessed: 2025-09-15.
- [8] Jan Bobolz, Jesus Diaz, and Markulf Kohlweiss. 2025. Foundations of Anonymous Signatures: Formal Definitions, Simplified Requirements, and a Construction Based on General Assumptions. In *Financial Cryptography and Data Security*, Jeremy Clark and Elaine Shi (Eds.). Springer Nature Switzerland, Cham, 121–139.
- [9] Dan Boneh and Chelsea Komlo. 2022. Threshold Signatures with Private Accountability. *Cryptology ePrint Archive*, Paper 2022/1636. doi:10.1007/978-3-031-15985-5_19
- [10] Benedikt Bünz, Lucianna Kiffer, Loi Luu, and Mahdi Zamani. 2019. Flyclient: Super-Light Clients for Cryptocurrencies. *Cryptology ePrint Archive*, Paper 2019/226. <https://eprint.iacr.org/2019/226>
- [11] Cardano Foundation. 2025. Cardano. <https://cardano.org/>. Accessed: 2025-09-15.
- [12] Cardano Foundation. 2025. Cardano Blockchain Explorer (Beta). <https://beta.explorer.cardano.org/>. Accessed: 2025-09-15.
- [13] Cardano Team. 2024. Full Cardano Ledger. <https://intersectmbo.github.io/formal-ledger-specifications/pdfs/cardano-ledger.pdf>.

- [14] Pyrros Chaidos and Aggelos Kiayias. 2025. Mithril: Stake-Based Threshold Multisignatures. In *Cryptography and Network Security*, Markulf Kohlweiss, Roberto Di Pietro, and Alastair Beresford (Eds.). Springer Nature Singapore, Singapore, 239–263.
- [15] Rutchathon Chairattana-Apirom, Lucjan Hanzlik, Julian Loss, Anna Lysyanskaya, and Benedikt Wagner. 2022. PI-cut-choo and friends: Compact blind signatures via parallel instance cut-and-choose and more. In *Annual International Cryptology Conference*. Springer, Springer, Cham, Switzerland, 3–31.
- [16] Manuel MT Chakravarty, Sandro Coretti, Matthias Fitzi, Peter Gazi, Philipp Kant, Aggelos Kiayias, and Alexander Russell. 2021. Fast isomorphic state channels. In *Financial Cryptography and Data Security: 25th International Conference, FC 2021, Virtual Event, March 1–5, 2021, Revised Selected Papers, Part II* 25. Springer, Springer, Cham, Switzerland, 339–358.
- [17] Manuel M. T. Chakravarty, James Chapman, Kenneth MacKenzie, Orestis Melkonian, Jann Müller, Michael Peyton Jones, Polina Vinogradova, Philip Wadler, and Joachim Zahnentferner. 2020. UTXO_{ma}: UTXO with Multi-asset Support. In *Leveraging Applications of Formal Methods, Verification and Validation: Applications - 9th International Symposium on Leveraging Applications of Formal Methods, ISOLa 2020, Rhodes, Greece, October 20–30, 2020, Proceedings, Part III (Lecture Notes in Computer Science, Vol. 12478)*, Tiziana Margaria and Bernhard Steffen (Eds.). Springer, Cham, Switzerland, 112–130. doi:10.1007/978-3-030-61467-6_9
- [18] Manuel M. T. Chakravarty, James Chapman, Kenneth MacKenzie, Orestis Melkonian, Jann Müller, Michael Peyton Jones, Polina Vinogradova, and Philip Wadler. 2020. Native Custom Tokens in the Extended UTXO Model. In *Leveraging Applications of Formal Methods, Verification and Validation: Applications: 9th International Symposium on Leveraging Applications of Formal Methods, ISOLa 2020, Rhodes, Greece, October 20–30, 2020, Proceedings, Part III (Rhodes, Greece)*. Springer-Verlag, Berlin, Heidelberg, 89–111. doi:10.1007/978-3-030-61467-6_7
- [19] Panagiotis Chatzigiannis, Foteini Baldimtsi, and Konstantinos Chalkias. 2021. SoK: Blockchain Light Clients. Cryptology ePrint Archive, Paper 2021/1657. <https://eprint.iacr.org/2021/1657>
- [20] David Chaum and Torben Pryds Pedersen. 1993. Wallet Databases with Observers. In *Advances in Cryptology — CRYPTO’ 92*, Ernest F. Brickell (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 89–105.
- [21] Adrian Brink Christopher Goes, Awa Sun Yin. 2022. Anoma : a unified architecture for full-stack decentralized applications. <https://github.com/anoma/whitepaper/blob/main/whitepaper.pdf>
- [22] Arthur Britto David Schwartz, Noah Youngs. 2018. The Ripple Protocol Consensus Algorithm. https://ripple.com/files/ripple_consensus_whitepaper.pdf
- [23] Bitcoin Developers. 2021. BIP 340: Schnorr Signatures for secp256k1. <https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki>. Accessed: 2025-05-28.
- [24] Ankur Dubey. 2023. A decentralised solver architecture for executing intents on EVM blockchain. <https://ethresear.ch/t/a-decentralised-solver-architecture-for-executing-intents-on-evm-blockchain/16608>. Accessed: 2025-05-28.
- [25] Taher ElGamal. 1985. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE transactions on information theory* 31, 4 (1985), 469–472.
- [26] Ergo Team. 2019. Ergo: A Resilient Platform For Contractual Money. <https://whitepaper.io/document/753/ergo-1-whitepaper>.
- [27] Georg Fuchsbauer and Mathias Wolf. 2024. Concurrently Secure Blind Schnorr Signatures. In *Advances in Cryptology – EUROCRYPT 2024*, Marc Joye and Gregor Leander (Eds.). Springer Nature Switzerland, Cham, 124–160.
- [28] Ariel Gabizon, Kostis Karantias, and Aggelos Kiayias. 2020. Plumo: Towards Scalable Interoperable Blockchains using zk-SNARKs. Cryptology ePrint Archive, Report 2020/399. <https://eprint.iacr.org/2020/399>
- [29] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. 2019. Plonk: Permutations over Lagrange-Bases for Oecumenical Noninteractive Arguments of Knowledge. Cryptology ePrint Archive, Report 2019/953. <https://eprint.iacr.org/2019/953>
- [30] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schofnegger. 2021. Poseidon: A New Hash Function for Zero-Knowledge Proof Systems. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Berkeley, CA, USA, 519–535. <https://www.usenix.org/conference/usenixsecurity21/presentation/grassi>
- [31] Jens Groth. 2016. On the Size of Pairing-based Non-interactive Arguments. Cryptology ePrint Archive, Paper 2016/260. <https://eprint.iacr.org/2016/260>
- [32] Lucjan Hanzlik, Julian Loss, and Benedikt Wagner. 2023. Rai-choo! Evolving blind signatures to the next level. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, Springer, Cham, Switzerland, 753–783.
- [33] Dominik Hartmann and Eike Kiltz. 2023. Limits in the Provable Security of ECDSA Signatures. Cryptology ePrint Archive, Paper 2023/914. <https://eprint.iacr.org/2023/914>
- [34] iden3. 2023. Circom: Circuit Compiler for Zero-Knowledge Proofs. <https://github.com/iden3/circom>. Accessed: 2025-05-28.
- [35] IntersectMBO. 2025. Cardano Ledger Specifications and Implementations. <https://github.com/IntersectMBO/cardano-ledger>. Accessed: 2025-05-28.
- [36] Simon Jentzsch and Christoph Jentzsch. 2018. EIP-1186: RPC-Method to get Merkle Proofs – eth_getProof. Technical Report 1186. Ethereum Improvement Proposals. Draft; accessed: 2025-09-15; available online: <https://eips.ethereum.org/EIPS/eip-1186>.
- [37] Joseph Poon and Thaddeus Dryja. 2010. *The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments*. Technical Report. University of Zurich, Department of Informatics.
- [38] Balazs Komuves. 2025. hash-circuits: Hashing circuits implemented in circom. <https://github.com/bkomuves/hash-circuits>. MIT License, maintained by Faulhorn Labs.

- [39] Yuan Lu, Qiang Tang, and Guiling Wang. 2020. Generic Superlight Client for Permissionless Blockchains. Cryptology ePrint Archive, Paper 2020/844. <https://eprint.iacr.org/2020/844>
- [40] Giulio Malavolta, Pedro Moreno-Sanchez, Aniket Kate, Matteo Maffei, and Srivatsan Ravi. 2017. Concurrency and Privacy with Payment-Channel Networks. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (Dallas, Texas, USA) (CCS '17)*. Association for Computing Machinery, New York, NY, USA, 455–471. doi:10.1145/3133956.3134096
- [41] Gregory Maxwell, Andrew Poelstra, Yannick Seurin, and Pieter Wuille. 2019. Simple Schnorr multi-signatures with applications to Bitcoin. *Des. Codes Cryptography* 87, 9 (Sept. 2019), 2139–2164. doi:10.1007/s10623-019-00608-x
- [42] S. Nakamoto. 2008. Bitcoin: A Peer-to-Peer Electronic Cash System. <https://bitcoin.org/en/bitcoin-paper>.
- [43] C. P. Schnorr. 1990. Efficient Identification and Signatures for Smart Cards. In *Advances in Cryptology — CRYPTO' 89 Proceedings*, Gilles Brassard (Ed.). Springer New York, New York, NY, 239–252.
- [44] Claus Peter Schnorr. 2001. Security of Blind Discrete Log Signatures against Interactive Attacks. In *Information and Communications Security*, Sihan Qing, Tatsuaiki Okamoto, and Jianying Zhou (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 1–12.
- [45] Ertem Nusret Tas, David Tse, Lei Yang, and Dionysis Zindros. 2024. Light Clients for Lazy Blockchains. arXiv:2203.15968 [cs.CR] <https://arxiv.org/abs/2203.15968>
- [46] Blockfrost Team. 2025. Blockfrost – Cardano API Platform. <https://blockfrost.dev/>. Accessed: 2025-09-15.
- [47] Ethereum Team. 2023. Ethereum development documentation. <https://ethereum.org/en/developers/docs/>
- [48] Ergo Platform Team. 2025. Ergo Platform. <https://ergoplatform.org/>. Accessed: 2025-09-15.
- [49] Khalani Team. 2025. Khalani Arcadia: An Open Platform for Intent-Driven Agent Collaboration. https://khalani.network/aip_whitepaper.pdf
- [50] Raiden Team. 2025. Raiden Network 3.0.1 Documentation. <https://raiden-network.readthedocs.io/>
- [51] Stefano Tessaro and Chenzhi Zhu. 2022. Short pairing-free blind signatures with exponential security. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, Springer, Cham, Switzerland, 782–811.
- [52] Psi Vesely, Kobi Gurkan, Michael Straka, Ariel Gabizon, Philipp Jovanovic, Georgios Konstantopoulos, Asa Oines, Marek Olszewski, and Eran Tromer. 2025. Plumo: An Ultralight Blockchain Client. <https://celo.org/papers/plumo> White paper.
- [53] VESPR. 2026. VESPR Wallet. <https://vespr.xyz/>. Accessed: 2025-10-14; Non-custodial light wallet for the Cardano network.
- [54] Polina Vinogradova, Orestis Melkonian, Philip Wadler, Manuel Chakravarty, Jacco Krijnen, Michael Peyton Jones, James Chapman, and Tudor Ferariu. 2024. Structured Contracts in the EUTxO Ledger Model. In *5th International Workshop on Formal Methods for Blockchains (FMBC 2024) (Open Access Series in Informatics (OASIs), Vol. 118)*, Bruno Bernardo and Diego Marmosler (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 10:1–10:19. doi:10.4230/OASIs.FMBC.2024.10
- [55] David Wagner. 2002. A Generalized Birthday Problem. In *Advances in Cryptology — CRYPTO 2002*, Moti Yung (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 288–304.
- [56] Barry WhiteHat, Jordi Baylina, and Marta Bellés. 2020. Baby Jubjub Elliptic Curve. Ethereum Improvement Proposal EIP-2494. <https://eips.ethereum.org/EIPS/eip-2494>
- [57] Pieter Wuille. 2012. BIP 0032: Hierarchical Deterministic Wallets. https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki#user-content-Full_wallet_sharing_m.
- [58] ScaleSphere Foundation Ltd. (“Foundation”). 2018. Celer Network: Bring Internet Scale to Every Blockchain. <https://celer.network/doc/CelerNetwork-Whitepaper.pdf>